

VIET NAM NATIONAL UNIVERSITY HO CHI MINH

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

Final Project

Super Mario Game

Course: Object-Oriented Programming - CS202

Student list:

Nguyễn Đình Minh Huy (25125083)

Trần Gia Huy (25125084)

Group 52 - Class 25A01

2nd September 2026



Contents

1	Abstract	1
2	Introduction and objectives	1
3	Group information and contributions	2
4	Requirements coverage against the rubric	2
5	Project architecture	4
6	OOP design	5
6.1	The entity tree	7
6.2	The other two branches: Item and Block	8
6.3	Characters, enemies and bosses	10
6.4	Player form: State plus Decorator	12
6.5	The pattern interfaces	13
6.6	Polymorphism where it earns its keep	14
6.7	Encapsulation	14
6.8	SOLID, principle by principle	15
7	Design patterns	19
7.1	Factory	20
7.2	Registry / Type Object	21
7.3	Singleton	22
7.4	State	23
7.5	Strategy	24
7.6	Template Method	25
7.7	Command — player input	26
7.8	Composite	27
7.9	Command with undo — the level editor	28
7.10	Command — the debug console	29
7.11	Observer	30

7.12	Decorator	31
7.13	Memento	32
7.14	Object Pool	34
7.15	Adapter	35
7.16	Two patterns deliberately not used	35
8	Implementation details	37
8.1	The frame	37
8.2	The physics pipeline	37
8.3	Input: commands, and why not <code>isKeyPressed</code>	38
8.4	Entities: one door in, one door out	39
8.5	Enemy behaviour and the boss template	39
8.6	The tile map and level loading	40
8.7	Time rewind, replay and the snapshot	40
8.8	The CPU opponent	40
8.9	Multiplayer	41
8.10	Graphics	41
8.11	The entity catalogue	41
8.12	Procedural generation, solvability and Endless Mode	42
9	Data storage and serialization	42
10	Technical problems and solutions	43
10.1	Collision: the ground was not solid	43
10.2	Gravity applied to things that should not fall	44
10.3	Object lifetime, four times	45
10.4	One fact, two places	45
10.5	Interaction that was never wired	46
10.6	A crash on Windows that could not be reproduced on macOS	47
10.7	A boss that could not be beaten, and a backdrop that took three tries	47
10.8	Process failures, and the rules they produced	48
10.9	Toolchain and porting	49

11 Features demonstration	49
12 Process and project history	53
12.1 Timeline	53
12.2 The procedure: checkpoint audits, repeated	54
12.3 What the audits produced	56
12.4 The rules came out of the failures	57
12.4.1 The audit corrects itself	58
13 Verification, CI and known gaps	59
13.1 Known gaps	59
13.1.1 Claims this project cannot verify with the environments it has	60
13.1.2 A defect neither fixed nor dismissed	60
13.1.3 Verification that stops short of observation	60
13.1.4 Performance left on the table, measured rather than guessed	61
13.1.5 Design trade-offs accepted rather than fixed	62
13.1.6 Specification items that were not built	62
14 Conclusion and future work	63
14.1 Future work: the far horizons	63
14.1.1 Learned agents: a trained policy behind the seam that already exists	64
14.1.2 Learned level generation, with the game as the oracle	65
14.1.3 One engine horizon: two players over a network	66
15 References	66
16 Appendix	67
16.1 Build and run	67
16.2 Controls	67
16.3 Figures at a glance	68
16.4 UML diagrams, collected	68

1 Abstract

This report documents a 2D side-scrolling platformer in the style of *Super Mario Bros.*, written in C++17 with SFML 3.0.2 and ImGui-SFML. The project's purpose is not the game as such but the architecture behind it: a four-level abstract inheritance tree over 13 enemy, 13 item, 10 block and 5 player classes, driven polymorphically by a fixed-timestep engine, and organised by ten software design patterns. The deliverable is 36,766 lines across 134 translation units and 145 headers, with 40 verification harnesses, 32 of which are registered with CTest and run by CI on every push.

Two things distinguish this report from a feature list. First, every capability claimed here has been checked to be **reachable from main() and observed running** — a class that compiles and a harness that constructs it are not evidence, and the project has a documented history of confusing the two. Second, §12 states plainly what is *not* done, including one rubric line the project deliberately forfeits.

2 Introduction and objectives

The assignment asks for a 2D Mario-style game demonstrating inheritance, encapsulation, polymorphism and abstraction, with at least five design patterns, three levels of increasing difficulty, collision detection, sound, state management and save/load. We proposed and were granted an expanded scope of 110 features, on the argument that a larger surface is what makes the patterns *load-bearing* rather than decorative: a Factory that builds three types is a switch statement with a nice name; one that builds 41 types, configured from JSON, is the reason the level loader does not know what a Goomba is.

Three engineering objectives shaped the result:

1. **No entity may know about the world.** An enemy that wants to throw a hammer cannot reach the entity list; it publishes an event. This one constraint is what keeps the dependency graph acyclic, and §10.1 describes what happened when the mechanism behind it was used carelessly.
2. **Behaviour is composed, not inherited.** Movement is a swappable strategy, player form is a state object, temporary power-ups are decorators around it. Adding "a Koopa that flies" is a constructor argument, not a subclass.

3. **Nothing is complete until it has been seen running.** Enforced by CI, and by a rule that a checkbox may not be ticked on the basis that a file compiles.

3 Group information and contributions

Work was split into two *vertical* domain slices rather than horizontal layers, so that both members wrote engine code and presentation code rather than one owning systems and the other owning UI.

Member	Domain	Systems work	Presentation work	Commits
A — Nguyễn Đình Minh 25125083	Player & World	Core engine and game loop, physics and collision pipeline, player entities and states, items, TileMap and level loading, camera, save/load, time rewind, map generator	Parallax backdrop, screen transitions, Menu / WorldMap / Playing / Options states, two-player modes, level editor	452
B — Trần Gia Huy 25125084	Enemies & Interaction	Input manager and command objects, sound manager, enemies and AI movement strategies, blocks, entity factory, object pool, replay recorder, debug console	Sprite sheets and animation, HUD, minimap, particles, death effects, audio wiring, character select, pause / game over / victory, statistics, achievements, boss fights	61

Commit counts are from `git shortlog -sne` on the integration branch and are a measure of activity, not of value: a large share of Member A's total is engine refactoring and defect work, which produces many small commits. Both members' work is present in every subsystem the game runs.

4 Requirements coverage against the rubric

Each row names where the capability lives and how it was confirmed. "Observed" means the game was run and the behaviour seen; "test" means a CTest case exercises it.

Rubric item	Status	Where it lives, and the evidence
Player inputs, movement, collision (20)	Done	<code>InputManager</code> + 8 command objects; <code>PhysicsEngine</code> with a fixed 1/60 s step, coyote time, jump buffering, wall slide, ground pound; <code>SpatialHash</code> broad phase into AABB narrow phase and <code>CollisionResolver</code> . Observed; covered by <code>verify_regressions</code> and <code>verify_all</code> .
Enemy behaviour (10)	Done	13 enemy classes including two bosses, driven by 8 interchangeable <code>IMovementStrategy</code> implementations (patrol, chase, fly, tethered chase, hammer throw, timer emergence, linear, proximity trigger). Observed; <code>verify_enemies</code> , <code>verify_enemies_new</code> .
Power-ups and items (10)	Done	13 item classes; five player forms as State objects plus two Decorators (Star, Mega). Observed; <code>verify_blocks</code> , <code>verify_regressions</code> .
Three levels (15)	Done	1-1 Grassland, 1-2 Ice Cavern, 1-3 Bowser's Castle, plus a bonus stage and three pipe-reachable sub-levels. All are JSON, loaded through <code>LevelLoader</code> ; 1-3 ends in a boss fight. Observed end to end.
Sound (10)	Done	29 effects and 12 music tracks. <code>SoundManager</code> subscribes to 20 event types, so audio is wired by the Observer bus rather than called from gameplay code. Observed; <code>verify_sound_visual</code> .
Object-oriented design (10)	Done	§6. Four-level abstract hierarchy, private state with action-oriented methods rather than trivial accessors, and polymorphic dispatch through <code>Entity*</code> in every engine pass.
Five design patterns (25)	Ten	§7 names all ten with their file and the problem each solves.
AI (5, additional)	Done	Two tiers. Per-enemy AI through movement strategies, and a full CPU <i>opponent</i> : <code>AIController</code> builds an observation of the world each frame and drives a second Player through the same command objects a human uses, with selectable skill and archetype. Observed — §11, figure 4.
Multiple players (5, additional)	Done	Four modes: Versus (two humans), Versus CPU, Co-op (shared lives and score, friendly fire off) and Shadow Chase (one human pursued by a 3-second-delayed replay of themselves). Independent key bindings per player; shared-screen camera framing the midpoint. Observed.
3D game (5, additional)	Not attempted	Deliberately forfeited. The brief offers 2D or 3D, and the project chose 2D: rebuilding the renderer in 3D would have consumed the effort that went into the pattern architecture without demonstrating any more OOP. We record this as a real 5-point cost rather than describing the parallax backdrop as "2.5D".

Rubric item	Status	Where it lives, and the evidence
Save / load (<i>requirement</i>)	Done	<code>Serializer</code> , JSON, three slots plus auto-save at checkpoints; separate files for settings, high scores and campaign progress. <code>verify_save_load</code> .
Game state management (<i>requirement</i>)	Done	Nine <code>IGameState</code> implementations on a stack-capable <code>GameStateManager</code> . <code>verify_frontend_states</code> .
Level editor (<i>bonus</i>)	Done	In-game ImGui editor (F1): full tile and entity palette, free camera, undo/redo via Command objects, JSON export/import. Observed — figure 5.
Character switching (<i>bonus</i>)	Done	Four playable characters with distinct physics (Luigi jumps $1.2\times$ higher, moves $0.85\times$, falls $0.9\times$, and double-jumps) and a selection screen.

5 Project architecture

The engine is layered, and the dependency arrows point one way only. Nothing in `Entities` includes anything from `Core` except the singletons and the event bus; nothing in `Physics` knows what a Goomba is.

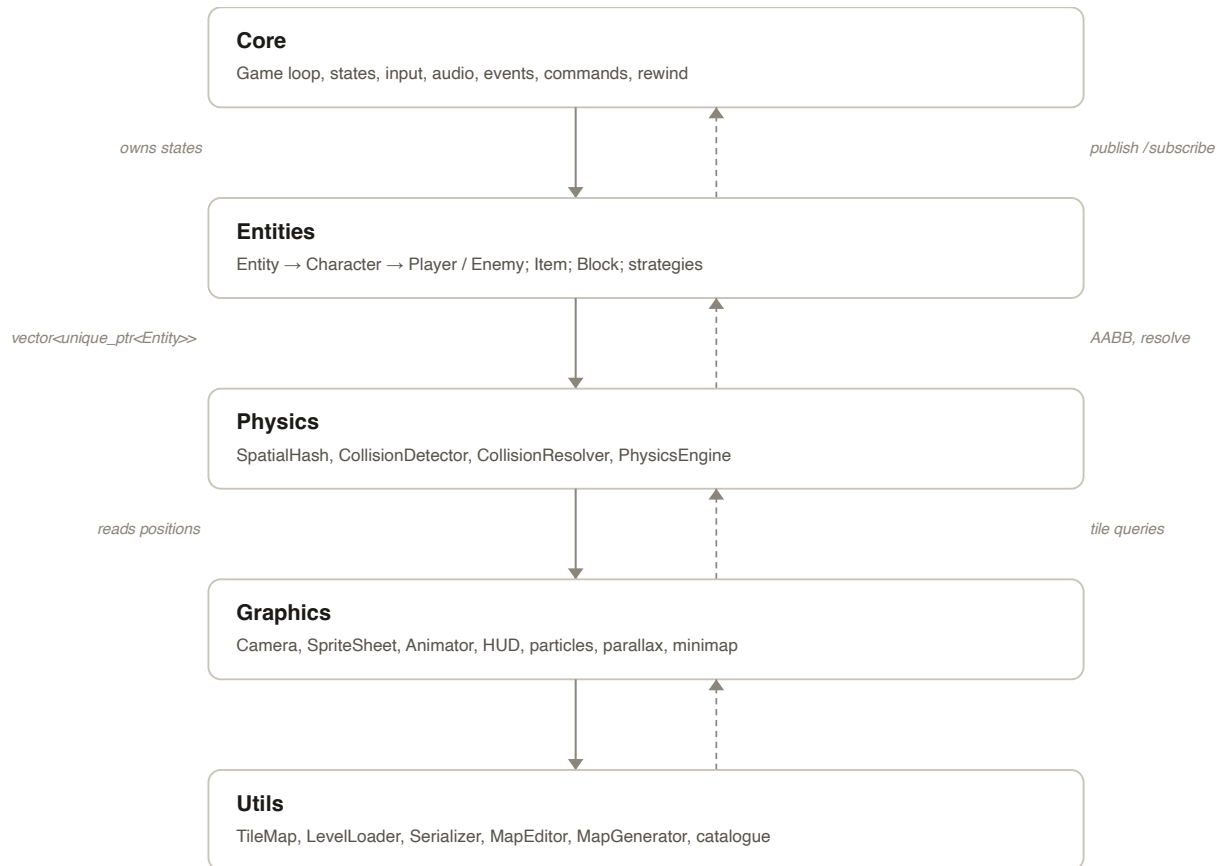


Figure 5 — The five-layer architecture.

Solid arrows are the ownership/data direction (Core owns the states that hold entities; entities are read as a flat list by physics; physics writes positions graphics then draws). Dashed arrows are the narrower channel each layer answers back through — an event, a resolved collision, a tile lookup — never a direct call in the other direction.

The frame is a fixed-timestep loop: input is drained from the OS event queue, the accumulator runs `update(1/60)` as many times as it owes, and rendering interpolates between the last two states. Physics is therefore deterministic and independent of frame rate, which is what makes the Memento-based time rewind (§7) reproducible.

6 OOP design

Every diagram in this section is **generated from the headers** by `SuperMarioGame/tools/gen_class_diagram.py` at the moment this report is built, so it cannot drift from the code. That is not a stylistic preference: the project's previous hand-written `class_diagram.md` had rotted

to the point of omitting `Boss`, `Bowser`, `BoomBoom`, `Spiny`, `Lakitu`, `ShadowMario`, `AIController`, `ObjectPool` and `TimeRewindManager` — most of the architecture worth drawing — while still reading as authoritative. It is now generated by the same script.

A dashed border and a `«stereotype»` tag mark an abstract class or an interface; the hollow triangle is UML generalization, pointing at the base.

6.1 The entity tree

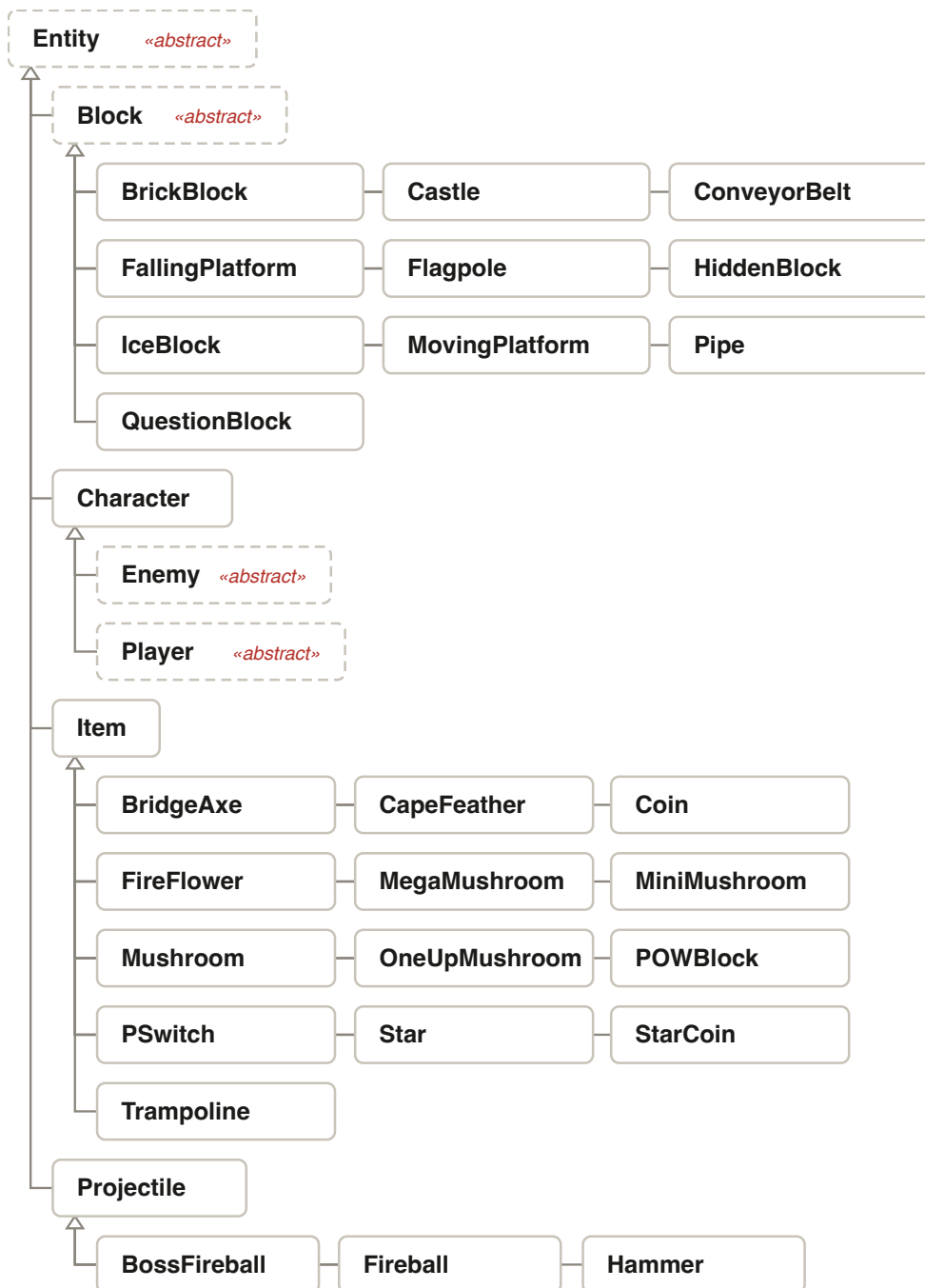


Figure 6 — The Entity hierarchy, two levels deep.

Four branches under one abstract root. Character adds motion state; Item adds activate/collect; Block adds `onHitFromBelow`; Projectile carries no gravity. The engine holds all of them as `Entity*`.

The root is abstract in the strict sense — `Entity::update`, `Entity::render` and `Entity::getBoundingBox` are pure virtual, so no leaf can exist without deciding what it does. The three second-level

abstracts each add exactly one obligation: `Item::activate(Player&)`, `Block::onHitFromBelow(Player&)`, and `Character`'s velocity and grounded state.

6.2 The other two branches: Item and Block

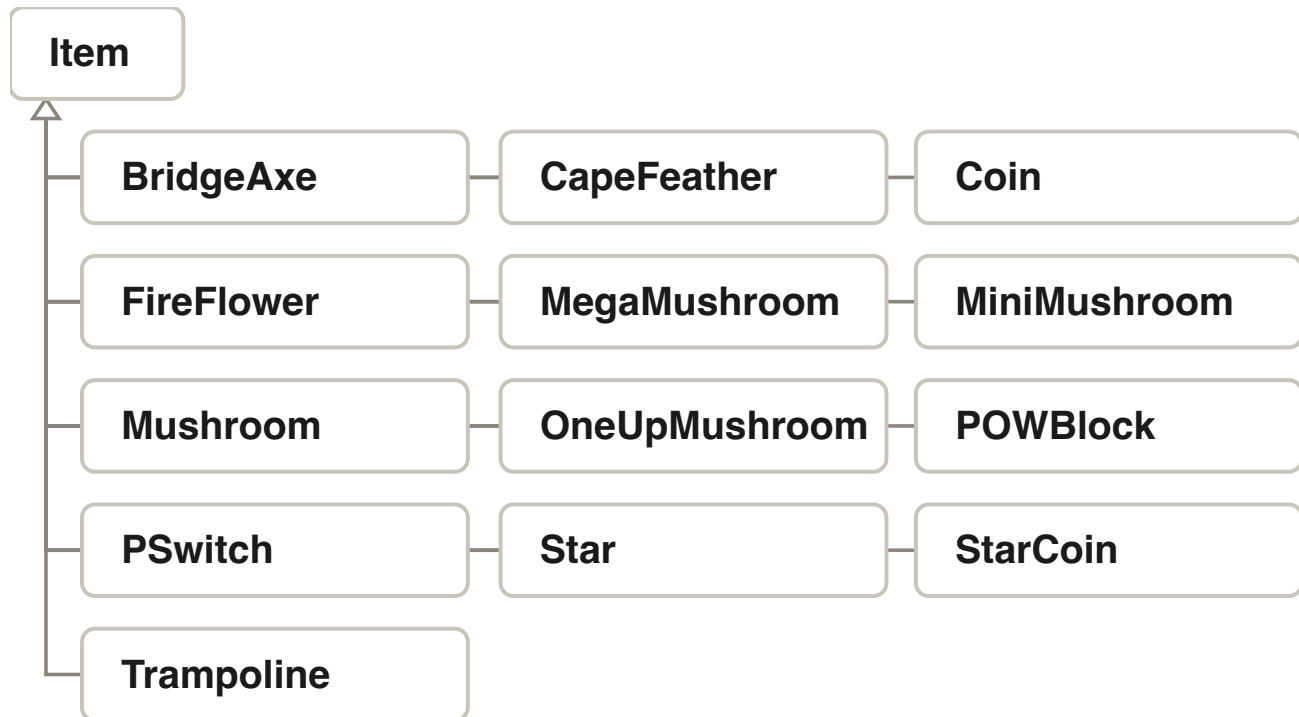


Figure 6a — Thirteen items behind one `activate()` call.

A Mushroom, a Star and a 1-Up all answer the same question — what happens to the player who touches me — with a different body. QuestionBlock never asks which one it is holding.

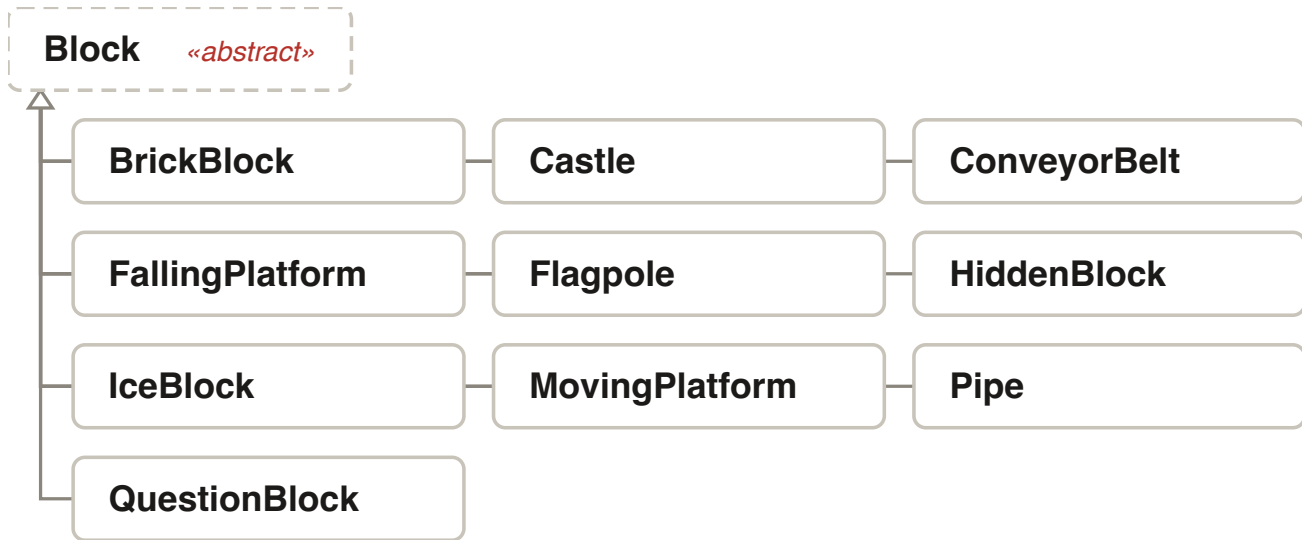


Figure 6b — Ten blocks, two of them stateful.

FallingPlatform and Thwomp each drive themselves through phases (Idle/Shaking/Falling/Respawning; wind-up/slam/rest/climb) held as a plain enum — state machines, but not State-pattern participants, which is argued in §7.4. Pipe and Flagpole are stateless: they fire once. Block does not know the difference, which is the point of putting it here rather than in each concrete class.

Item and **Block** look similar — both are collidable, both are mostly inert until the player reaches them — and the project’s own history has a concrete argument for keeping them as separate branches rather than merging them into one "InteractableEntity". A block’s defining question is *what is on the other side of this collision, structurally* (`collidesWithTiles()`, `onHitFromBelow`) — the physics engine needs to know before it resolves a frame. An item’s defining question is *what happens to the player who reaches it* (`activate(Player&)`) — physics does not need to know in advance, because touching a Star does not change how the world resolves collisions this frame. Collapsing the two would have handed the physics engine’s collision-resolution code a branch on "is this actually solid" for every entity that is nominally a block, which is exactly the kind of type-testing OOP’s dispatch is supposed to replace. **PSwitch** is the edge case that proves the split is real work and not just naming: it is a **Block** (grounded, collidable) whose `onHitFromBelow` triggers a level-wide event rather than reacting locally — still a block, because the physics engine still needs to resolve a jump-bump against it, but the one that most resembles an item.

Ten concrete blocks and thirteen concrete items ship in this project, more than twice what the rubric’s five-item minimum would need per SPEC’s rubric-alignment reasoning (§2) — the same

doubled-scope argument the feature list makes, made visible in a diagram instead of a count.

6.3 Characters, enemies and bosses

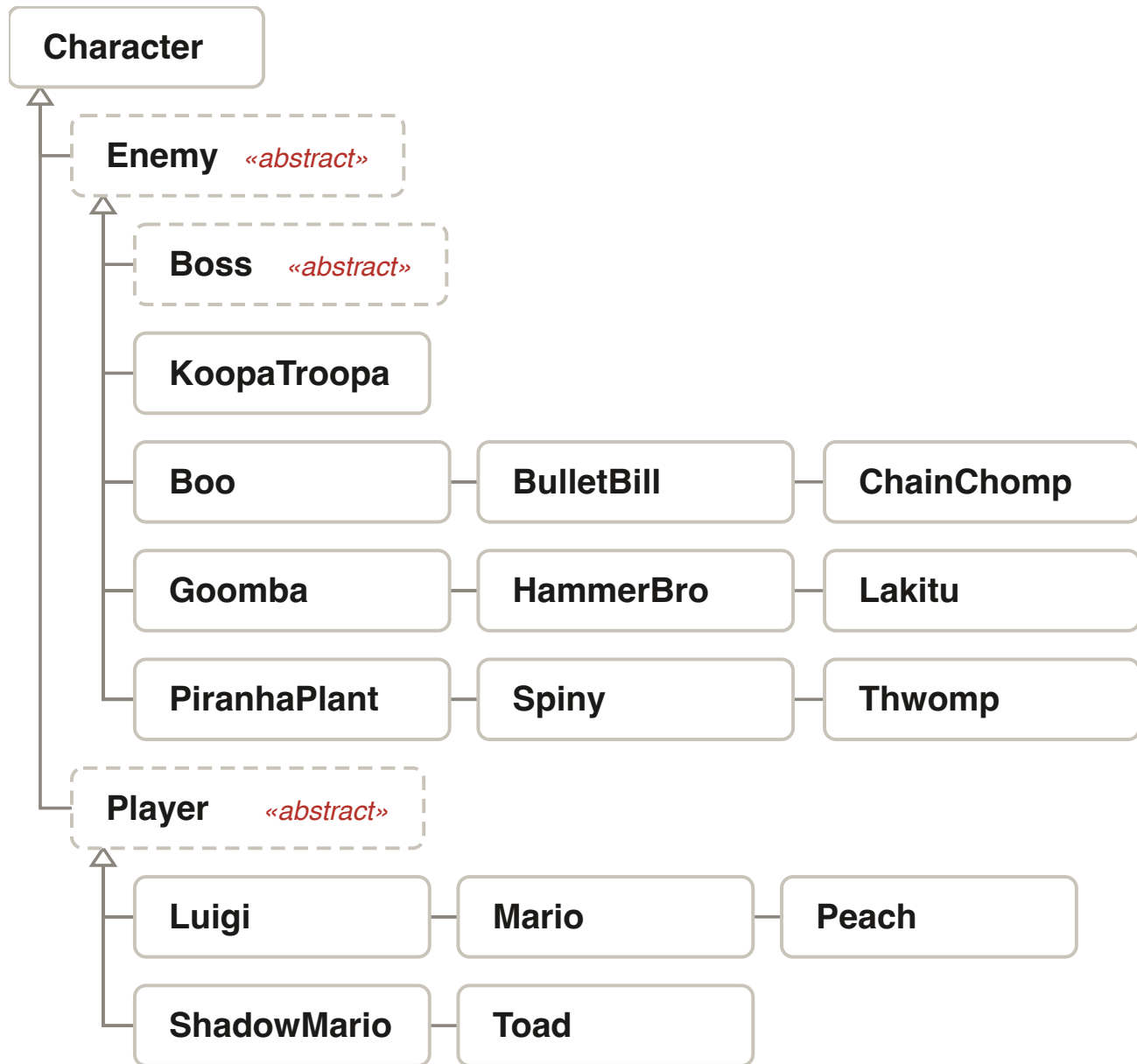


Figure 7 — Character splits into Player and Enemy.

Five playable characters including ShadowMario, the delayed replay of the human player used by Shadow Chase mode. Enemy is where the movement strategy is held.

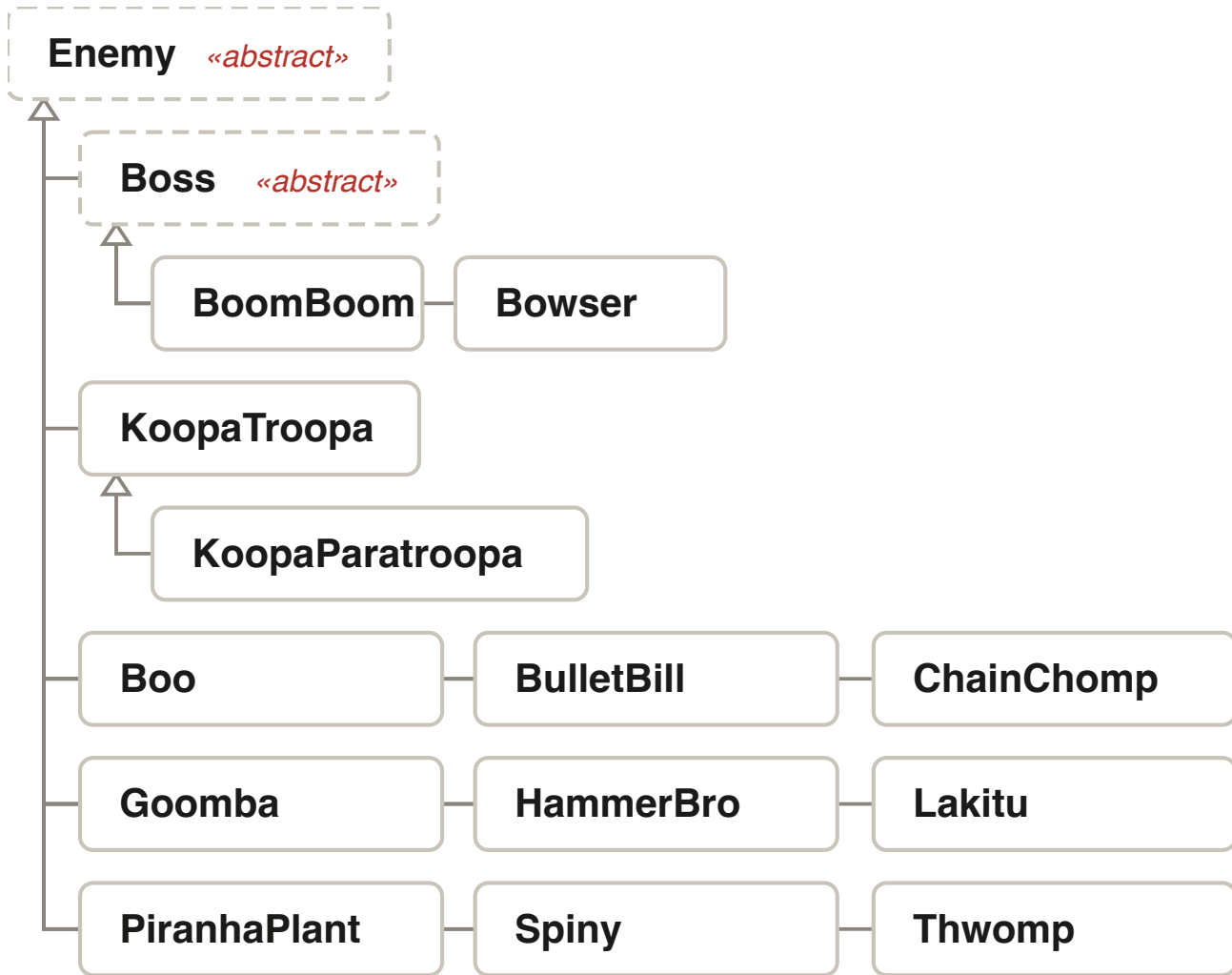


Figure 8 — The enemy branch, with the Boss sub-hierarchy.

Boss inserts a whole layer between Enemy and its two concrete fights: health bar, phases, i-frames, arena and stagger window, sequenced by a final update() so no boss can skip them.

Boss is the clearest argument in the codebase for a deep hierarchy rather than a flag. It is not "an enemy with more health": it seals `update()` as `final` and calls a pure-virtual `updateBehaviour()`, so *every* boss gets its invulnerability frames, phase transitions and defeat sequence in the correct order and a new one writes only its attack pattern. Bowser adds fire breath, a phase-2 leap and the fireball-stagger window; BoomBoom adds a charge. Neither can forget the rest, because neither is given the chance.

6.4 Player form: State plus Decorator

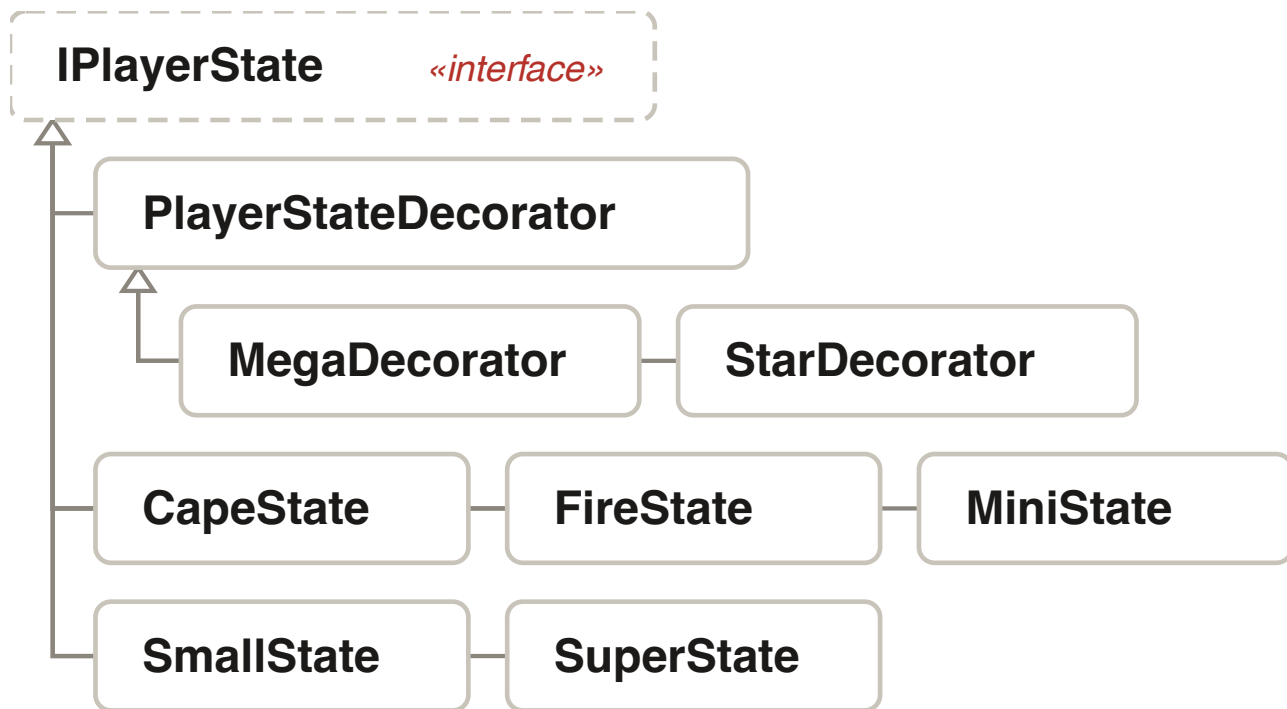


Figure 9 — Five forms and two decorators.

PlayerStateDecorator wraps whatever base state is active, which is what lets a Star be temporary and orthogonal: Fire Mario with a Star is still Fire Mario underneath.

Form changes Mario's jump height, hitbox and reaction to damage. As five state objects plus a transition table, that lives in one place; as booleans it was a growing pile of conditionals. The decorators are the part worth noticing — a sixth "Star state" would have to remember which form to exit back into, whereas wrapping preserves it for free.

6.5 The pattern interfaces



Figure 10 — Nine screens behind one interface (State).

GameStateManager holds the stack as a vector, not a `std::stack`, so `render()` can walk down through an overlay: *PauseState* draws over *PlayingState* rather than replacing it.

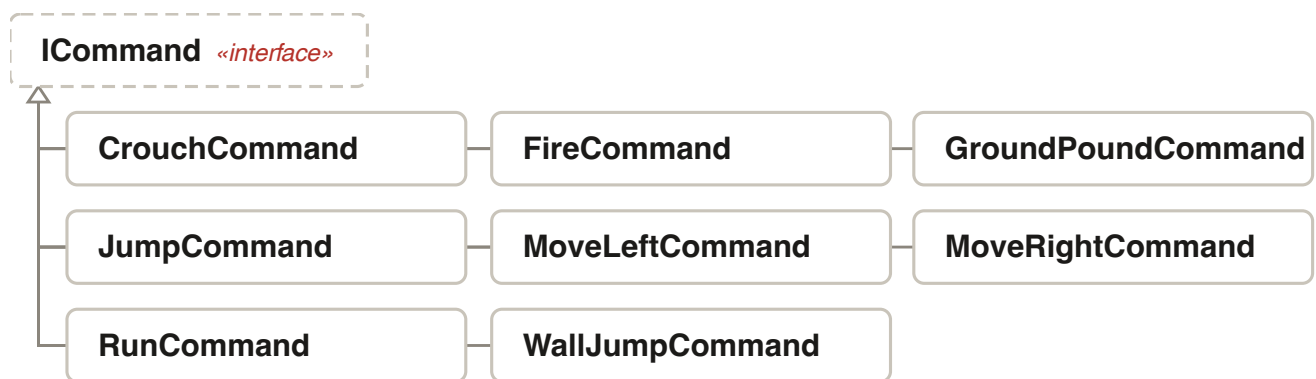


Figure 11 — Input as objects (Command).

Every action the player can take is an object, which is what makes key rebinding possible and lets a second player use the same commands through a different binding table.



Figure 12 — Eight interchangeable movements (Strategy).

A Koopa and a Paratroopa differ only in which of these they hold, so "the same enemy but flying" is a constructor argument rather than a subclass.

6.6 Polymorphism where it earns its keep

The engine never asks what an entity is. `PhysicsEngine::update` takes `vector<unique_ptr<Entity>>` and calls virtual methods: `getGravityMultiplier()` returns 0 for a block and 1 for a Goomba, so one integrator handles both; `collidesWithTiles()` returns false for a dying player, which is what lets a corpse fall through the floor without a single special case in the physics code. The one place a `dynamic_cast` chain used to exist — a 30-branch function turning an entity into its serialised name — was replaced by a virtual `getTypeName()`. That change also fixed a live defect: the chain tested base classes before derived ones, so several types were shadowed by their parent and saved to disk under the wrong name.

6.7 Encapsulation

State is private or protected and reached through action-oriented methods: `player.takeDamage(1)` rather than a settable health field; `boss.tryStomp()`, which returns whether the hit actually landed, rather than a public health counter the resolver would have to interpret. Where a getter exists it answers a question the caller has a right to ask (`isInvulnerable()`, `isStaggered()`), never handing out an internal container or a raw window pointer.

Two deliberate exceptions, both narrow. The physics classes are declared `friend` of `Entity` so they can write positions during resolution — preferred over public position setters that any code could reach. And `Serializer`'s file-path helpers stay private even though the tests wanted them: the tests were given `Serializer::clearHighScores()` instead, because what they needed was the table cleared, not the path it lives at.

Two properties of the whole tree are worth stating because they can be checked rather than asserted. First, **no class in the project exposes a mutable data member**: every declared field sits behind `private` or `protected`, and the only public data anywhere are `struct` aggregates whose whole purpose is to carry values across a boundary (`GameSnapshot`, `PlayerSnapshot`, `EntitySnapshot`, `EntityConfigEntry`, `EntityCatalogue::Entry`) — a simplification argued for in §7.13 and

charged as a cost there. The one getter that broke the rule is worth recording because of how it was found and closed rather than because it survives: `Camera::getView()` used to return a mutable `sf::View&`, handing any caller the camera's internals. The audit written for this submission sweep flagged it as the class's only encapsulation leak, and it is now `const sf::View& getView() const`, with the single caller that genuinely had to reshape the view — `EditorState`, fitting the canvas rectangle — served by two named operations, `Camera::setViewSize` and `Camera::setViewViewport`. That is the shape the fix should take: not a wider getter, but the narrowest operation the real caller needs.

Second, **ownership is uniform**. Every owning pointer in the project's own code is a `std::unique_ptr`, and there is not a single `delete` anywhere in `src/` or `include/` — the eight textual matches are all the word appearing in comments or in `= delete`. Raw `new` now occurs exactly once, in the deliberately never-deleted `ResourceManager` instance of §7.3, whose 27-line rationale explains why. It had occurred eight times: the other seven were spawner lambdas in `DevPanel` that wrapped a raw `new` in a `unique_ptr` on the same line, and they now call `EntityFactory::create` instead, which closed a raw allocation and an open/closed violation in one edit. Shared ownership appears exactly once in the project's own code, for `ICommand`, and §7.8 explains why.

6.8 SOLID, principle by principle

The five principles are taken one at a time below, each with the strongest evidence the codebase can offer *and* the place where it is knowingly not followed. The second half of each paragraph is the more useful one: a principle stated without its exception is a slogan, and every exception here was decided rather than drifted into.

Single responsibility. The evidence for is structural: physics does not know what a Goomba is, entities do not resolve their own collisions, `EntityCatalogue` is the single declaration of what an entity type *is*, and the class that owns a fight (`Boss`) is separate from the class that owns a level. The evidence against is one file. `PlayingState` is a god class: its `.cpp` is past four thousand lines and its header past eight hundred, and it orchestrates level loading, spawning, physics stepping, the camera, HUD synchronisation, boss arenas, two-player rules, time rewind, pipe transitions, cheats, the editor bridge and endless-mode chunking. The next largest source file in the project is under a quarter of its size. **The decision was to document it rather than split it in submission**

week, and the reason is that the direction has already been demonstrated: `DevPanel`, `DebugCheats`, `LightingRenderer`, `PipeRenderer` and the `EditorBridge` port were all carved out of this class, each extraction verified by playing the game afterwards. Five more extractions in the last week, each touching the one class every mode runs through, would put the working build at risk to improve a number in a report. The trend is the argument; the remaining size is the honest cost, and it is listed in §13 as work rather than as an achievement.

Open/closed. This is the principle the project can demonstrate mechanically rather than assert. Adding an entity type is *one row* in `EntityCatalogue` — the factory, the level parser and the editor palette all read that table, and `verify_r21_entity_registry` walks every enumerator from 0 to `EntityType::Count` and fails the suite for any row that is missing, so the openness is enforced by a test rather than by a convention. The same shape holds on four more axes: a new enemy behaviour is one `IMovementStrategy`, a new difficulty one `IDifficultyStrategy`, a new screen one `IGameState`, a new undoable editor operation one `IEditorCommand` — and none of them requires editing a `switch`. Where it is not held: run-time type dispatch survives in three places, each named with its reason. The decorator chain of §7.12 is inspectable only by `dynamic_cast`, at seventeen sites, which is an inherent cost of that pattern rather than a lapse. Per-type serialization in `Serializer` and `LevelLoader` still branches on type, because the on-disk schema is a compatibility surface that deliberately does not follow the class hierarchy. The third had no defence and so was fixed rather than argued for: `PlayingState::recycleEntity` used to run three sequential `dynamic_casts` to decide which `ObjectPool` an expiring projectile belonged to. It now switches on `Entity::poolTag()`, a virtual returning an `Entity::PoolTag` that `Fireball`, `Hammer` and `BossFireball` each override — one v-table lookup instead of three RTTI walks, and a fourth pooled type needs an override rather than another cast block. That is the difference this section is trying to draw: two of the three remaining type tests are the price of a decision, and the third was simply a lapse, which is why only the third was removed.

Liskov substitution. The evidence is that the engine's hot paths never ask what an entity is. `PhysicsEngine::update` takes a `std::vector<std::unique_ptr<Entity>>` and integrates all of it through virtuals: `getGravityMultiplier()` returns 0 for a block and 1 for a Goomba, so one integrator handles both, and `collidesWithTiles()` returns false for a dying player, which is what lets a corpse fall through the floor with no special case anywhere in the physics code. The sharpest measurement is in `CollisionResolver`: identifying both sides of a contact once cost up to twelve

sequential `dynamic_casts` per colliding pair per frame, and the file now contains **zero** — the four textual matches left in it are comments recording the change. Contact rules are virtual hooks (`onStomped`, `onHitFromBelow`, `onPlayerTouch`) over an ordered category pair, and every hierarchy root in the project — `Entity`, `IGameState`, `ICommand`, `IMovementStrategy`, `IPlayerState`, `IEditorCommand`, `IConsoleCommand`, `IAIPolicy`, `IEntityAdmitter`, `IDifficultyStrategy` — declares a virtual destructor, so deleting through a base pointer is always defined. `PlayerStateDecorator` is the cleanest case of the principle: it substitutes for `IPlayerState` by forwarding every operation to what it wraps, which is exactly why a `Star` can be layered over any form. Two honest qualifications. Four of the five player forms satisfy their interface partly by *doing nothing* (§7.4), so their substitutability is cheap rather than earned. And `EntityFactory::createUnconfigured` may return `nullptr`, a weakened postcondition every caller has to check — accepted, because level files are hand-editable and a bad name must not be fatal.

Interface segregation. The interfaces added most recently are the narrowest, which is the direction to want. `IEntityAdmitter` has two methods and a contract written into the header saying what each must guarantee; `IEditorCommand` has three; `IAIPolicy` has three, one of them defaulted. `IGameState` asks five things every screen genuinely does and puts the rest — `isOverlay`, `onSuspend`, `onResume` — behind defaults, so a screen implements only what it needs. `IMovementStrategy` is segregated in a second sense: the public surface is `execute` plus three defaulted queries, while the three sequencing hooks are `protected`, so a caller cannot reach a phase and a subclass cannot reorder them. Three failures, all real. `ICommand::execute(Character&)` is a narrow interface with too fat a parameter and no argument channel, which is precisely why the debug console needed a second hierarchy (§7.10). `IPlayerState` demands five methods of implementers that mostly need one. And the widest interface in the codebase is not an interface at all: `Entity` and `Character` each declare **twelve** friend classes — `PhysicsEngine`, `CollisionResolver`, `PlayingState`, and then `IMovementStrategy` together with all eight of its concrete strategies. That list is the bill for §7.5: moving movement out of the entity means the thing that moves it is no longer the entity, and it still has to write velocity. Friendship was chosen over public position and velocity setters, because a setter is reachable by *every* caller while a friend list is at least enumerable and reviewable — but nine of the twelve entries are one pattern's cost, and the smaller fix (a single `protected` mutation API the strategies inherit access to) is recorded in §13 rather than claimed here.

Dependency inversion. Where a dependency crosses a layer boundary, it is inverted through

an interface, and the examples are the load-bearing ones: `PhysicsEngine` and `CollisionResolver` depend on `Entity` and on `EntityCategory`, never on `Goomba`; `GameStateManager` depends on `IGameState`; `MapEditor` depends on `IEditorCommand`; the editor's entity commands depend on `IEntityAdmitter` rather than on `PlayingState`, which is the whole point of §7.15; `AIController` depends on `IAIPolicy`; and `Game` consults `IDifficultyStrategy` instead of comparing a string. Then the cost, owned rather than minimised. The specification says this project has four singletons. It has **twelve**, and they are reached from everywhere: `Game::getInstance()` appears at 143 call sites in 39 files, `SoundManager::getInstance()` at 86 in 31, `EventBus::getInstance()` at 54 in 28. Every one of those is a dependency that does not appear in a constructor signature and cannot be substituted for a fake, which is the textbook DIP failure and is stated here as such. The reason it was chosen is real but partial: SFML's audio device, texture cache and event bus are genuinely process-wide, and a Meyers singleton gives a defined construction order where a file-scope global would load a texture before the graphics context exists. What that does *not* excuse is the number. `ScreenTransitionManager` and `EntityDeathEffect` each have exactly one caller outside their own implementation file — `PlayingState` — so both are singletons by habit rather than by necessity, and each would be better as a member of whatever owns it. Injecting twelve managers through the constructors of a class the size of `PlayingState` is a week's work carrying live regression risk, so the decision was the same as for single responsibility: document the number, keep the dependencies that were *dangerous* under control, and put the injection work in §13 where a reader can see it is known rather than missed. The dangerous ones were the lifetime hazards, and each now has a named mechanism: `EventBus::ScopedSubscription` for subscriber lifetime, which `SoundManager` was the last holdout against — its twenty subscriptions were raw ids that never unsubscribed, and are now a `std::vector<EventBus::ScopedSubscription>`, closing the last place where a callback could outlive its owner by omission rather than by design; a constant-initialised `g_eventBusAlive` flag so a `ScopedSubscription` destructor running after the bus is gone cannot resurrect a destroyed singleton; and the written rationale above `ResourceManager`'s never-deleted instance, which explains why leaking one fixed-size allocation at process exit is the correct answer to an undefined static destruction order. The pattern across all four of those is worth naming, because it is the honest answer to a reader who counts twelve singletons and stops there: the number was not reduced, and the failure modes it creates were closed one at a time, each with a mechanism a compiler or a destructor enforces rather than a rule someone has to remember.

7 Design patterns

The rubric asks for five patterns. The specification claims ten. This section takes each of the ten in turn, adds the four the codebase grew without ever claiming them, and names the two it deliberately does *without* — because a pattern that was considered and rejected for a stated reason is better evidence of design than a pattern that happens to be present.

Every subsection answers the same four questions in the same order, and the order is the argument: **the problem** this codebase actually had, **the naive alternative** that was in the code or would have been, **why this pattern** answers it, and **what it cost** — because a pattern with no cost has not been used for anything. Participants are named with the file they live in, so every claim here can be checked against the tree rather than taken on trust.

§	Pattern	Root participant	Claimed?
7.1	Factory	EntityFactory::create	yes
7.2	Registry / Type Object	EntityCatalogue::Entry	<i>no</i>
7.3	Singleton	12 managers	yes
7.4	State	IGameState, IPlayerState	yes
7.5	Strategy	IMovementStrategy	partly
7.6	Template Method	Boss::update	yes
7.7	Command (input)	ICommand	yes
7.8	Composite	CompositeCommand	<i>no</i>
7.9	Command, undoable	IEditorCommand	<i>no</i>
7.10	Command (console)	IConsoleCommand	partly
7.11	Observer	EventBus	yes
7.12	Decorator	PlayerStateDecorator	yes
7.13	Memento	GameSnapshot	yes
7.14	Object Pool	ObjectPool<T>	yes
7.15	Adapter	IEntityAdmitter	<i>no</i>
7.16	Rejected on purpose	Visitor, Flyweight	—

A note on figures before the patterns themselves. The class diagrams in §6 and §16.4 are generated from the headers by `gen_class_diagram.py`, which draws *inheritance* edges. A pattern whose structure *is* a hierarchy therefore has a figure and is pointed at below; a pattern whose structure is an association — Factory, Registry, Observer, Memento, Object Pool, Adapter — has no generated figure, because there is no generalization arrow for the tool to find. Two hierarchies that do exist

are also missing from the figures for a duller reason: `CompositeCommand` and the eleven `IConsoleCommand` classes are declared inside `.cpp` files, and the generator only scans headers. That is recorded as a known gap in §13 rather than papered over with a hand-drawn diagram that would then rot.

7.1 Factory

The problem. Levels are JSON. `LevelLoader` reads the string `"koopaparatroopa"` out of a file a human may have hand-edited, and has to end up holding a `std::unique_ptr<Entity>` without knowing that a `Koopaparatroopa` class exists. So do `MapGenerator` (which builds endless chunks at runtime), `PlaceEntityCommand` (the editor's brush), `PlayingState::spawnProjectile` (anything asked for at run time through an `EntitySpawnRequested` event) and `DevPanel`'s seven spawn buttons: five callers, one question.

The naive alternative. Each caller includes every concrete entity header and switches on the type: `if (name == "goomba") return new Goomba(pos); else if ....` It is fine for ten types. At the 41 concrete entity classes this project ships it is a 41-branch function that has to be edited in *five* places for every new entity, in files whose real job is reading a file, generating terrain, painting tiles, stepping a frame and drawing a debug panel respectively. That is not hypothetical: it is what the code did, and §7.2 records what it cost. `DevPanel` was the last caller still doing it by hand — seven `new X(p)` lambdas naming seven concrete item classes — and it was routed through the factory during this submission sweep.

Why this pattern. `EntityFactory::create(EntityType, sf::Vector2f)` is the single construction point. Callers ask a question — build me this — instead of making a decision. And because construction is funnelled, the factory can insert a step no caller knows about: `applyConfig()` looks the new entity's `getTypeName()` up in `assets/config/entities.json` and overrides speed and score value where the file has an opinion. Balance tuning became a data edit without a single call site changing, which is the payoff a factory is actually for — not saving a `switch`, but owning a policy step.

What it cost. Two things, both real. Construction is now indirect, so a stack trace from a broken constructor passes through a function pointer rather than naming the caller. And the factory returns `nullptr` for a type it cannot build instead of failing to compile: the compile-time exhaustiveness a `switch` over an enum would have given up is gone. That cost is paid for deliberately — level files

are hand-editable, so a bad name must not crash the game — and bought back by a test rather than by the compiler (§7.2).

Participants. `EntityFactory` (creator), `EntityCatalogue::Entry::create` (the concrete creators), `EntityConfig` / `EntityConfigEntry` (the tuning policy), `Entity` and its subclasses (products). This is a *Simple Factory with a registry*, not GoF Factory Method: there is no hierarchy of creators, and deliberately so — a creator subclass per entity type would have doubled the class count to remove a table.

7.2 Registry / Type Object

The problem. This is the pattern the specification never claimed and the one with the sharpest evidence, because the bug it fixed was invisible. The same list of entity types was written out by hand in four places: `EntityFactory::create` knew 40 types, `SerializationUtils` knew 40 names, the `MapEditor` palette knew 16 — none of them enemies — and `PlaceEntityCommand`'s if-chain knew 16 name strings and *built nothing*. The consequence: the level editor, whose entire purpose is placing entities, could not place a Goomba, a Koopa, a pipe, a question block or a flagpole. Clicking the button did nothing and nothing anywhere reported a failure.

The naive alternative. Fix the four lists and remember to edit four files next time. That is what was actually tried first, and then guarded with a regression test that compared the factory switch against the table — which is a guard on a hazard, not the removal of one.

Why this pattern. `EntityCatalogue::Entry` makes the type itself an object: one row per entity type carrying its `EntityType` enumerator, its canonical serialised `name`, the `label` the editor shows a human, its palette `Category`, and — the field that turns a description into a registry — a `Creator` function pointer that builds one. The parser, the palette and the factory all read this one table. Adding an entity type is one row. The factory's 42-case `switch` is gone, and `createUnconfigured` is two lines: look the entry up, call its creator.

What it cost. The compile-time exhaustiveness of §7.1, bought back as a test: `verify_r21_entity_registry` walks every enumerator from 0 to `EntityType::Count` and fails the suite for any that has no row, so a type added to the enum and forgotten cannot become a palette button that silently places nothing. A function pointer rather than `std::function` keeps the storage cost at zero — every creator is a capture-less lambda — and, more usefully, cannot

be left empty in a way that would fail only at the call site. The three types needing more than a position (a moving platform's travel range, two projectiles' launch velocities) supply it inside their own lambda, which is the honest wrinkle: the uniform creator signature is uniform because three rows hide a constant inside themselves.

Participants. `EntityCatalogue::Entry`, `EntityCatalogue::all()`, `findByName` / `findByType`, `EntityCatalogue::Category`; `verify_r21_entity_registry` as the mechanism that keeps it total. The data-driven half of the same idea is `EntityConfigEntry`, where negative and empty fields mean *the file did not say* so a half-filled JSON entry cannot silently zero a value the constructor had set.

7.3 Singleton

The problem. There is exactly one audio device, one texture cache and one event bus in a process, and the objects that need them are everywhere: an `Item` deep in the entity vector wants to publish an event; a state being constructed wants a font. Handing each of them a reference would mean threading three parameters through every constructor in the entity tree.

The naive alternative. File-scope globals. Those are initialised before `main()` in an order the standard does not define across translation units, which for an SFML texture cache means loading a texture before the graphics context exists.

Why this pattern. Meyers singletons — a function-local `static` returned by `getInstance()` — are constructed on first use, so the order is defined by the program's own call sequence and nothing is built before `main()`. Copy and move are deleted on each, so the single instance is enforced by the compiler rather than by convention.

What it cost. This is the trade-off the report is least comfortable with, and it is stated rather than hidden. There are **twelve** of them, not the four the specification claims: `Game`, `SoundManager`, `EventBus`, `InputManager`, `ResourceManager`, `AchievementManager`, `ScreenTransitionManager`, `StatisticsTracker`, `DebugConsole`, `ReplayRecorder`, `ParticleSystem` and `EntityDeathEffect`. `Game::getInstance()` alone appears at 143 call sites across 39 files. Every one of those is a dependency that does not appear in a constructor signature and cannot be substituted in a test — the dependency-inversion cost is discussed as a decision in §6.8. One of the twelve is a deliberate exception worth naming: `ResourceManager`'s instance is a heap allocation that is never deleted,

because destruction order between this translation unit's statics and SFML's is not something the program gets to choose. It is one fixed-size allocation reported as *still reachable* rather than lost, and the real run still frees everything deterministically through `Game::shutdown()` calling `clear()` while the window is alive. The 27-line comment above it exists so a later reader does not "fix" it.

7.4 State

The problem. Two independent instances of the same problem. At screen level: pausing must draw the level underneath the pause menu, and a state must be able to ask for its own removal. At player level: Mario's hitbox, jump behaviour and reaction to damage all change with his form.

The naive alternative. An enum `m_screen` plus a switch in `handleInput`, `update` and `render`; and a pile of booleans on `Player`. The enum cannot express "paused *over* playing" at all — before the stack existed, `render()` drew only the top of the stack, so pushing anything hid the game entirely.

Why this pattern. `IGameState` has five pure virtuals (`enter`, `exit`, `handleInput`, `update`, `render`) and **nine** concrete screens: `MenuState`, `CharacterSelectState`, `WorldMapState`, `PlayingState`, `PauseState`, `OptionsState`, `GameOverState`, `VictoryState` and `EditorState`. Nine, not the ten the specification lists: there is no `StatisticsState` class, because the statistics screen is a *page* of `OptionsState` (`OptionsState::Page::Statistics`, alongside `Settings`, `Controls`, `HighScores` and `Achievements`) rather than a state of its own — which is the right call, since those five pages share one navigation model and pushing five states to get five pages would be the pattern applied for its own sake. `GameStateManager` holds the nine in a `std::vector` rather than a `std::stack` precisely so `render()` can walk *down* through overlays: `isOverlay()` returning true means "I do not own the screen, keep drawing what is beneath me". `IPlayerState` is the second axis, with five concrete forms (`SmallState`, `SuperState`, `FireState`, `CapeState`, `MiniState`).

What it cost. Three costs, and the third is a design criticism of our own code. First, nine classes and a heap allocation per transition. Second, the transitions had to become *deferred*: `pushState`, `popState` and `changeState` queue a `PendingOp` applied at a frame boundary, because a state that pops itself would otherwise be destroyed while its own member function was still on the stack; `IPlayerState::isExpired()` exists for the same reason on the player axis, so a timed state reports expiry instead of swapping itself out from inside `update()`. Third and most honestly: **four of the**

five player forms have empty method bodies. Only `CapeState` carries behaviour — the glide, and the cape spin. The other four differ by their `getSize()` return value alone. The State axis on the player is paid for in five classes and under-used; the behaviour that *does* vary lives in the Decorator layer above it (§7.12). It is kept because `CapeState` proves the axis is the right shape and because form-specific behaviour has somewhere obvious to go — but a reader should know that today it is mostly a size table with a virtual interface on it.

Not State participants. The specification lists `FallingPlatform` and `Thwomp` as State participants. They are not. Each drives itself through a plain `enum class` and switches on it: `FallingPlatformState` is `Idle/Shaking/Falling/Respawning`, and the `Thwomp`'s phases live in its strategy as `ProximityState` — `Idle/Slamming/Resting/Rising`. Both are state *machines*, which is a technique, not the State pattern, which is a set of polymorphic objects. Four phases in one class do not earn four classes, and promoting an `enum` to a hierarchy to raise a pattern count would be the wrong trade — naming both here is the more useful answer than counting them. Turning the `Thwomp`'s bare `int` into a named `enum class` did fix a real defect, though: it had been picking its own sprite from `position.y > 140.0f` instead of asking the machine what it was doing.

Participants and figures. `IGameState` and its nine screens — Figure 10; `GameStateManager` (context, no figure: it holds states rather than deriving from anything). `IPlayerState` and its five forms — Figure 9.

7.5 Strategy

The problem. A Koopa Troopa and a Koopa Paratroopa are the same enemy with a different way of moving — and a stomped Paratroopa is supposed to *become* a walking Koopa, at runtime, mid-frame.

The naive alternative. `switch (m_enemyType)` inside `Enemy::update`, which puts every enemy's movement in one function and makes "the same enemy but flying" a new subclass. Runtime conversion under that design means destroying one object and constructing another, at the moment something is holding a pointer to it.

Why this pattern. `Enemy` holds a `std::unique_ptr<IMovementStrategy> m_aiStrategy` and delegates. There are **eight** implementations — `PatrolStrategy`, `ChaseStrategy`, `FlyStrategy`, `LinearStrategy`, `TimerEmergenceStrategy`, `HammerThrowStrategy`, `TetheredChaseStrategy` and

ProximityTriggerStrategy — and "flying" is a constructor argument: KoopaParatroopa takes a FlyStrategy and, when stomped, calls `setStrategy(std::make_unique<PatrolStrategy>(true, false))`. The object survives; only its behaviour is replaced. The specification also lists a SwimStrategy: **it does not exist**, and swimming is descoped — eight, not nine.

The same pattern on two more axes. IDifficultyStrategy has three implementations (EasyDifficulty, NormalDifficulty, HardDifficulty) answering four questions with different numbers: enemySpeedScale, startingLives, levelTimeScale, bossHealthScale. Its history is the argument for it: the difficulty string had been saved, persisted and edited by the options screen since save/load was written, and *read by nothing* — picking Hard changed a word in `config.json`. Turning it into a strategy turned a dead string into four consulted numbers without spreading a difficulty switch across the level loader, the player and the bosses. And IAIPolicy::decide(const AIObservation&) → AIAction is the same seam for the CPU opponent, with AIController sensing and actuating around it. Reported honestly: it has **one** implementation today, HeuristicPolicy. A one-implementation interface is not yet a Strategy earning its keep; it is a seam placed where a learned policy would plug in, and §14 says so rather than counting it as a fourth axis.

What it cost. Per-enemy state migrated *into* the strategy objects, and that leaked. A PiranhaPlant's pipe mouth and a ChainChomp's post are anchors held by the strategy, so when the endless-mode chunk translation moved an enemy, its strategy dragged it straight back to the old anchor on the next tick. The fix was a new virtual, IMovementStrategy::translateAnchor(sf::Vector2f), defaulted to do nothing — asked of the strategy rather than cast for by the enemy, because which strategies hold an anchor is the strategies' own business. That is the recurring cost of delegating behaviour: the interface grows a method every time the outside world needs to know something the strategy is now hiding.

Participants and figure. IMovementStrategy (strategy), Enemy (context), the eight concretes — Figure 12. IDifficultyStrategy and IAIPolicy have no generated figure; both are association-held interfaces.

7.6 Template Method

The problem. Two places where a sequence must not be got wrong. Every movement strategy has to sense, then move, then clamp — and clamping before moving produces an enemy that walks

through walls for one frame. Every boss has to run its invulnerability frames, phase transitions and defeat sequence in the right order.

The naive alternative. Each of the eight strategies writes its own `execute()`, and each of the two bosses writes its own `update()` — which means copying `Bowser::update` into `BoomBoom` and editing the attack pattern in place. That is exactly how a second boss gets built without this pattern, and exactly how an i-frame check quietly diverges between two copies over a few edits.

Why this pattern. `IMovementStrategy::execute(Enemy&, float)` is *non-virtual* and its whole body is three calls: `calculateTarget` → `applyMovement` → `checkConstraints`. Only `applyMovement` is pure virtual; the other two are protected hooks defaulted to nothing, so `LinearStrategy` overrides one and `TetheredChaseStrategy` overrides all three, and neither can reorder the phases. `Boss::update(float dt)` is declared `final` and calls a pure-virtual `updateBehaviour(float)`: sealing it is a compiler-enforced "do not copy this function", so a third boss *can only* be added by writing its attack pattern.

What it cost. Rigidity, which is the point and also the price. A strategy that needs a fourth phase — something after clamping — has nowhere to put it and must either abuse `checkConstraints` or change the skeleton for all eight. Sealing `Boss::update` has the same shape of cost: a boss that genuinely needed to run its behaviour before the i-frame check cannot, and would force the base class to grow a hook.

Participants and figures. `IMovementStrategy` with `calculateTarget/applyMovement/checkConstraints` — Figure 12 shows the hooks and their visibility. `Boss` → `Bowser`, `BoomBoom` — Figure 8.

7.7 Command — player input

The problem. Keys must be rebindable from the options screen, and a second player has to trigger the same actions from a different key.

The naive alternative. `if (sf::Keyboard::isKeyPressed(Key::W)) player.jump();` scattered through the update loop. Rebinding then means finding every literal key code in the codebase, and a second player means duplicating all of them.

Why this pattern. `ICommand` declares one operation, `execute(Character&)`, and each action is an object: `JumpCommand`, `MoveLeftCommand`, `MoveRightCommand`, `FireCommand`, `RunCommand`, `Crou`

`chCommand`, `GroundPoundCommand`, `WallJumpCommand`. `InputManager` keeps an action table (`m_commandsByAction`: "jump", "fire", "left" ...) and per-player key maps, so `applyBindings()` can move a command to a different key without knowing how it was constructed, and player 2's map points at the very same command objects.

What it cost. An indirection between a key press and a jump, so a movement bug is now read in two files rather than one. And the interface is narrow in the wrong direction: it takes a whole `Character&` but carries no arguments, which is why the debug console could not reuse it and needed a second hierarchy (§7.10).

What this Command is *not*. Two claims in the specification are corrected here. `ICommand` has **no undo** — input commands are fire and forget; undo lives in a different hierarchy, §7.9. And it has **no serialization**: the replay system does not record commands, it records snapshots, which makes replay a Memento (§7.13), not a Command. Saying so is more useful than the claim, because the reason is a real engineering constraint — see §7.13.

Participants and figure. `ICommand` (command), `InputManager` (invoker), `Character` (receiver), the eight concretes — Figure 11.

7.8 Composite

The problem. One key has to do two things. Pressing jump should perform an ordinary jump, or a wall jump when the player is against a wall — and which one applies is decided by the commands themselves, not by the input layer.

The naive alternative. Give `InputManager` a special case: bind jump to `JumpCommand` and, at the call site, also invoke `WallJumpCommand`. The input layer then knows that two particular commands are related, which is precisely the knowledge the Command pattern was introduced to remove from it.

Why this pattern. `CompositeCommand` derives from `ICommand` and holds a `std::vector<std::shared_ptr<ICommand>>`; its `execute` forwards to each child in order. A composite *is* a command, so the binding table stores it in the same slot as a leaf and `applyBindings()` cannot tell the difference. Rebinding jump rebinds the pair.

What it cost. `ICommand` had to be held by `std::shared_ptr` rather than `std::unique_ptr`, because the same leaf command is referenced by both the action table and a composite. `std::`

`shared_ptr<ICommand>` in `InputManager` is the *only* shared ownership in the project's own code — every other owning pointer is a `std::unique_ptr` — so this pattern is the single exception to the ownership rule §6.8 argues from, and it is named rather than glossed. And `CompositeCommand` is declared in `InputManager.cpp`, so no generated figure can show it.

Participants. `CompositeCommand` (composite), `JumpCommand` and `WallJumpCommand` (leaves), `ICommand` (component).

7.9 Command with undo — the level editor

The problem. A level editor without undo is unusable, and this one has to undo two different kinds of change: tiles in a `TileMap` and entities in a *live* `PlayingState`'s entity vector.

The naive alternative. Snapshot the whole level before every edit and restore the last snapshot on `Ctrl+Z`. For a 200-tile-wide map with an entity list that is being simulated, that is a large copy per brush stroke — and it cannot describe what it is about to undo, so the editor could never show a history.

Why this pattern. `IEditorCommand` declares three operations — `execute()`, `undo()`, `describe()` — and each of the **seven** concretes stores its own inverse at construction, not the world's state: `PlaceTileCommand` and `EraseTileCommand` remember the `m_oldType` they overwrote, `FillRectCommand` and the rectangle it painted over, `MoveEntityCommand` the position it came from, `SetEntityPropertyCommand` the previous value, and `PlaceEntityCommand` / `EraseEntityCommand` the entity identity. `MapEditor` then needs nothing clever: two vectors, `m_undoStack` and `m_redoStack`, with `undo()` moving the top of one to the other. `describe()` returns one present-tense line ("Place Goomba") which is what makes the editor's History panel possible — a snapshot scheme has nothing to print. Bound to `Ctrl+Z` and `Ctrl+Shift+Z` in `EditorState::handleInput`, and to the same two menu items.

Why this is the stronger Command. The specification claims undo for the input commands, where there is none. It exists here, and this is where undo is *worth* having: an editor is the one part of the program whose user expects to take an action back. `FillRectCommand` is the clearest evidence that the granularity is a design decision rather than an accident — painting a 20×10 region a tile at a time put 200 entries on the stack, so undoing one rectangle meant pressing `Ctrl+Z` two hundred times. One command, one rectangle, one undo. The same reasoning made a drag into a single Mo

`veEntityCommand` capturing where the drag began, rather than one command per frame of mouse movement.

What it cost. Every editing operation must be expressible as an object with an inverse, which rules out an edit whose inverse is not knowable — and it forced a second interface into existence. `PlaceEntityCommand` used to push straight into the entity vector and `EraseEntityCommand` destroyed whatever it was handed; both bypassed the state that owns the vector, which is the only thing that knows an entity needs `setupAnimations()` before it can draw anything but a coloured placeholder box, and that `PlayingState::m_player`, `Game::setPlayer` and `InputManager::registerPlayer` hold raw pointers into it. That is §7.15.

Participants. `IEditorCommand` (command), the seven concretes, `MapEditor` (invoker and history), `TileMap` and `IEntityAdmitter` (receivers), `EditorState` (client). No generated figure: `IEditorCommand` is not yet one of the diagram tool's roots (§13).

7.10 Command — the debug console

The problem. A developer types `spawn bowser` or `give star` into an in-game console and expects text back, including an error message when the argument is wrong.

The naive alternative. One long `if/else` over the typed verb inside the console's own render function, with the help text written out separately — two lists to keep in step, which is the failure §7.2 already paid for once.

Why a *separate* interface. `ICommand::execute(Character&)` cannot express this: a console command takes an argument list, acts on the game as a whole rather than on one character, and *returns* text. So `IConsoleCommand` declares `name()`, `help()` and `execute(const std::vector<std::string>&) → std::string`, and there are **eleven** concretes: `help`, `give`, `lives`, `teleport`, `god`, `spawn`, `difficulty`, `level`, `progress`, `replay`, `clear`. The `help` command is a loop over `DebugConsole::commandNames()` rather than a hand-written list, and a command whose arguments are wrong answers by returning its own `help()` line — so a command and its usage text cannot drift apart, which is the same one-fact-two-places failure §7.2 paid for once already.

What it cost. A second Command hierarchy rather than one, which is the honest consequence of `ICommand`'s signature having been designed for exactly one use. Errors are returned as text rather than thrown, deliberately: a typo in a console is not exceptional. And every console command

reaches the world through the singletons of §7.3 — which is what keeps the console independent of whichever state is on top, and simultaneously the clearest example of the dependency cost in §6.8.

Participants. `IConsoleCommand`, its eleven concretes, `DebugConsole` (invoker and registry). Declared inside `DebugConsole.cpp`, so no generated figure can reach them.

7.11 Observer

The problem. Collecting a coin has to update the score, play a sound, advance the statistics counters and possibly unlock an achievement. Defeating a boss has to do a different four things. Neither the `Coin` nor the `Bowser` class has any business knowing what those systems are.

The naive alternative. `Coin::activate(Player& p)` calling `p.addCoins(1); hud.refresh(); soundManager.play("coin"); stats.recordCoin(); achievements.checkCoinMilestones()` ; — and then `Star::activate`, `OneUpMushroom::activate` and every other item repeating some subset of the same list, because each item's author has to remember which systems care this time.

Why this pattern. `EventBus::publish(const GameEvent&)` takes an `EventType` and a `std::any` payload; subscribers register a `std::function` against a type. There are **29** event types today. The real subscribers, counted from the code rather than from the specification, are `SoundManager` (20 subscriptions), `PlayingState` (15), `AchievementManager` (9), `Camera` (7), `StatisticsTracker` (4) and `Minimap` (1). A seventh listener — a future daily-challenge coin counter — subscribes without `Coin.cpp` changing at all. The bus also solved a structural problem the naive version cannot: an entity has no handle on the world's entity list, so `EntitySpawnRequested` with an `EntitySpawnRequest` payload is how Lakitu drops Spinies and Hammer Bro throws hammers — the requester never touches the vector.

Correcting the specification. `Hud` is listed as a subscriber and **subscribes to nothing**. It is push-fed instead: `PlayingState` assembles a `HudData` struct and hands it over through `Hud::sync` once per frame, so the HUD is a pure renderer of state given to it and holds no reference to a player or a bus at all. That is a defensible design for something which redraws unconditionally every frame — a subscription would buy it nothing — but it is *not* Observer and is not counted as one here. `ComboTracker` and `AchievementTracker` appear in the specification's participant list and **do not exist** as classes; combos are tracked inside `PlayingState` and achievements by `AchievementManager`.

What it cost. Three costs, all paid. Control flow is untraceable at the call site: a reader of `Coin::activate` can see that something was published and cannot see what happens next, which is a real loss of local reasoning in exchange for a real gain in decoupling. The `std::any` payload moves a type error from compile time to run time. And subscriber lifetime had to be engineered rather than assumed — a callback that outlives its object is a use-after-free the next time that event fires. Three mechanisms answer that: cancelled subscriptions are *tombstoned* rather than erased, so a handler that unsubscribes during delivery cannot invalidate the vector being walked; an `m_publishDepth` counter defers compaction while `publish()` is on the stack, including re-entrantly; and `EventBus::ScopedSubscription` is a move-only RAI handle that unsubscribes in its destructor, so a token cannot be forgotten. `PlayingState` holds fifteen of them, and `SoundManager`'s twenty — which were raw ids that never unsubscribed at all, the last such holdout in the codebase — are now a `std::vector` of them. That third mechanism is a pattern in its own right, RAI scope-bound resource management, and it is here for a reason worth stating: counting on thirty-five hand-written unsubscribes across two destructors is counting on a person, and this bus had already been given an alive-flag guard precisely because someone once did not write one.

Participants. `EventBus` (subject), `EventType / GameEvent` (the event), `EventBus::Callback` subscribers, `EventBus::ScopedSubscription` (lifetime). No generated figure: subscribers relate to the bus by association, not inheritance.

7.12 Decorator

The problem. A Star makes the player invincible for ten seconds; a Mega Mushroom makes him giant for eight. Both are *temporary* and both are *orthogonal to form* — Fire Mario with a Star is still Fire Mario, and has to still be Fire Mario when the Star runs out.

The naive alternative. Add `StarState` and `MegaState` to the five forms of §7.4. Each then has to remember which form to exit back into, and the two together are a cross-product: starred-and-mega-and-fire is a state nobody wrote. The cruder alternative is worse still — booleans on `Player` (`isSuper`, `isFire`, `isCape`, `isMini`, `isStarred`, `isMega`) with an if chain in every place form matters, each chain having to be taught the combination separately.

Why this pattern. `PlayerStateDecorator` derives from `IPlayerState` and *holds* a `std::unique_ptr<IPlayerState>`, forwarding `enter`, `exit`, `handleInput`, `update` and `getSize` to

whatever it wraps. `StarDecorator` adds a countdown and invincibility; `MegaDecorator` adds a countdown and *scales the wrapped state's* `getSize()` rather than returning a constant, which is what makes "giant Fire Mario" fall out instead of being designed. When a decorator expires, `releaseWrappedState()` hands the inner state back and the player is exactly what he was. The combination was never a special case: it is a consequence of the two axes being orthogonal.

What it cost. The chain is only inspectable by `dynamic_cast`, and there are **seventeen** such sites across `Player`, `Serializer`, `CollisionDetector` and `DevPanel`. The sharpest is `Player::setBaseState`, which must *hand-walk* the chain: cast the current state to `PlayerStateDecorator*`, loop inward through `getWrappedState()` until the cast fails, then swap the base form underneath the innermost decorator — which is what lets a Fire Flower picked up during a Star survive the Star expiring. That is a genuine cost of the pattern, not a bug: a decorator chain deliberately hides its depth, so any code that needs to reach *through* it has to ask at run time. An alternative would be a `getInnermost()` virtual on `IPlayerState` — which widens the interface every implementer must satisfy in order to serve two call sites, and was rejected on that basis.

Honest scope note. `StarDecorator` is exercised in normal play: four shipped question blocks carry `QuestionBlock::Content::Star`, one each in `level_1`, `level_2`, `level_3` and `bonus_1`. `MegaDecorator` is *not*: no shipped level places a Mega Mushroom, so it is reachable only through the debug console's `give` command or the level editor. The same is true of `MiniState` from §7.4 — no level's question block carries `MiniMushroom` either. All three are production code paths rather than harness-only ones, but a grader who only plays the campaign will meet the Star and neither of the other two, and the honest place to fix that is a level edit rather than a paragraph.

Participants and figure. `IPlayerState` (component), `SmallState...MiniState` (concrete components), `PlayerStateDecorator` (decorator), `StarDecorator` and `MegaDecorator` (concrete decorators) — Figure 9, which shows the wrapping member alongside the inheritance edge.

7.13 Memento

The problem. Holding R rewinds the last five seconds of play. Attract mode replays a recorded run on the menu screen. Both need the world's past.

The naive alternative. Record the player's inputs and re-simulate. That is the usual way and it was rejected for a stated reason: it requires the simulation to be bit-for-bit deterministic, and this

one is not — float physics, an entity list that spawns and prunes, and enemy strategies that read a shared `Game` singleton. Storing state costs more disk and plays back exactly what happened.

Why this pattern. `GameSnapshot` is the memento: `PlayerSnapshot` for each player, a `std::vector<EntitySnapshot>`, the level timer and the camera centre. `PlayingState` is the originator, building one per frame; `TimeRewindManager` is the caretaker, holding a `std::deque` capped at 300 frames — five seconds at 60 fps. `ReplayRecorder` reuses the *same* memento rather than inventing a second mechanism: a replay is that snapshot stream kept for longer, thinned to every Nth frame, and written to disk. The load-bearing detail is that `EntitySnapshot` keys on `Entity::getId()` and **not** on a position in the entity vector, because indices shift whenever an entity is pruned or spawned between record and restore — which is a defect that actually shipped: rewind assigned positions to the wrong entities.

What it cost. Two things, stated plainly. First, this is a *partial* snapshot, not "full game state": per entity it captures id, position, velocity and active flag, and per player position, velocity, score, coins and lives. Anything else — an enemy's strategy phase, a block's broken state, a decorator's remaining time — is not rewound. That is a deliberate trade for a per-frame capture cost, and the report should not be read as claiming more. Second, `GameSnapshot` is a fully public `struct`, so the originator's internals are readable by anything that can see the header — the narrow-interface half of the GoF pattern is absent. That is the common C++ simplification (a memento as an aggregate) and it is a real encapsulation cost; making the fields private with `PlayingState` and `Player` as friends would close it and add to the friend count §6.8 already argues about. Even the partial capture has been wrong once in a way worth recording: it captured player 1 only, so rewinding a two-player match rolled one player's score back and left the other's alone — silent, because nothing on screen looks wrong until you read the numbers. `hasSecondPlayer` and `secondPlayerState` exist because of it.

Participants. `GameSnapshot`, `PlayerSnapshot`, `EntitySnapshot` (mementos), `PlayingState` (originator), `TimeRewindManager` and `ReplayRecorder` (caretakers). No generated figure: these are aggregates and holders, with no inheritance among them.

7.14 Object Pool

The problem. Fireballs, hammers and boss fireballs are created and destroyed several times a second during a fight, each one a heap allocation inside the frame budget.

The naive alternative. The textbook pool owns its objects and hands out raw pointers. That does not fit this game at all: `PlayingState` owns the world as `std::vector<std::unique_ptr<Entity>>` and prunes inactive entries every frame, so a pool that kept ownership would mean rewriting how every entity in the game is stored.

Why this pattern, shaped this way. `ObjectPool<T>` trades in `std::unique_ptr<T>::acquire()` hands one over, `release()` takes it back, and in between the object is owned exactly the way an unpooled one would be. The entity list does not know pooling exists; the only change at the call site is that the prune step offers spent objects back instead of dropping them. `acquire()` forwards its arguments to `T`'s constructor on a miss and to `T::resetForPool` on a hit, so both paths leave the caller holding the object it asked for — and only `T` knows what "just-constructed again" means, which is why that requirement is on `T` rather than in the pool.

What it cost. A pool that grows without bound is a leak that never frees, so `m_maxRetained` caps the free list and `release()` simply lets the object die past the cap. The larger cost is structural and shows how a pattern leaks: recycling means the *owner* has to know which pool an expiring entity belongs to, even though pooling was supposed to be invisible to the entity list. `PlayingState::recycleEntity` answered that with three sequential `dynamic_casts` — exactly the run-time type test §6.8 argues against elsewhere, and indefensible rather than a trade-off, so it was fixed rather than written up. `Entity` now carries a virtual `poolTag()` returning an `Entity::PoolTag`, which `Fireball`, `Hammer` and `BossFireball` override; recycling switches on the answer. The residual cost is honest and small: `Entity`, the root of the whole hierarchy, now knows that pooling exists, which it did not before. That is a real widening of the base class in exchange for removing three RTTI walks per expiring projectile, and it is the trade this subsection would defend if asked.

Honest scope. Three types are pooled: `ObjectPool<Fireball>`, `ObjectPool<Hammer>` and `ObjectPool<BossFireball>`. The specification also claims particles and Bullet Bills. Neither is true: the particle system does its own recycling over a flag array, and no `ObjectPool<BulletBill>` exists. Three, not five.

Participants. `ObjectPool<T>` (pool), the three pooled projectile classes with their `resetForPool`

1, `PlayingState` (client, via `spawnProjectile` and `recycleEntity`). No generated figure: the pool is a template held by composition.

7.15 Adapter

The problem. An editor command has to add an entity to, or remove one from, a *live* game state — but it must not depend on `PlayingState`, and there are two facts it cannot know: that an entity which has not had `setupAnimations()` called on it renders as a flat coloured placeholder box, and that three separate places hold raw non-owning pointers into the entity vector.

The naive alternative. What the code did: `PlaceEntityCommand` pushed straight into the vector, so every entity the editor placed drew as a coloured box; and `EraseEntityCommand` destroyed whatever it was handed, so erasing a `Player` left `PlayingState::m_player`, `Game::setPlayer` and `InputManager::registerPlayer` all dangling.

Why this pattern. `IEntityAdmitter` is a two-method port — `admit(Entity*)` and `release(Entity*)`, neither taking ownership — with a contract stated in the header: `admit` must leave the entity fully drawable, `release` must have dropped every observer pointer into it before it returns. `PlayingState::EditorBridge` implements it and is the adapter: it presents the state's own knowledge through an interface the editor can depend on without depending on the state.

What it cost. One more interface and one more indirection for what is, in the common case, a `push_back`. It buys back two shipped defects and, more importantly, it makes the requirement *writable* — the invariant now lives in a header contract rather than in a maintainer's memory.

Participants. `IEntityAdmitter` (target), `PlayingState::EditorBridge` (adapter), `PlayingState` (adaptee), `PlaceEntityCommand` / `EraseEntityCommand` (clients). No generated figure: the implementer is a nested class, which the diagram tool does not index.

7.16 Two patterns deliberately not used

These belong in a design section as much as the fifteen subsections above, because each was the obvious answer to a real problem in this codebase and each lost for a reason that can be stated.

Visitor, and double dispatch generally. `CollisionResolver` must decide what happens when two entities touch, which is a two-argument dispatch C++ does not give you. The textbook answers are Visitor — an `accept/visit` pair on `Entity` — or a chain of `dynamic_casts`. The chain is what

the code had: up to twelve sequential casts per colliding pair per frame. Visitor was rejected too, on two grounds. It would put a `visitPlayer/visitEnemy/visitItem/visitBlock/visitProjectile` surface on a base class that has no other reason to know those categories exist, and it makes the *entity* hierarchy closed instead of the operation: adding a new collision *rule* would be cheap, but the resolver is stable and the entity list is not, so Visitor optimises the axis that does not change here. What the code does instead is ask each side once: `Entity::getCategory()` returns an `EntityCategory`, and the resolver switches on the *ordered pair* of the two categories, halving the case count because `(Enemy, Player)` is handled as `(Player, Enemy)` with the collision normal flipped. Two virtual calls and one switch replaced twelve casts. The enum is deliberately a *category* and not a type id — it answers "how does this collide?", which is the only question the resolver asks; anything needing the concrete type is told to add a virtual of its own instead of widening the enum. The result is measurable: the resolver contains **zero** real `dynamic_cast`, and the four textual matches in that file are comments recording this decision.

Flyweight. It would be easy, and wrong, to call `ResourceManager` and `SpriteSheet` a Flyweight and claim one more pattern. Flyweight splits an object's intrinsic state from extrinsic state passed in at use, so that many fine-grained objects can share one instance. What these two do is simpler and different: they are a **cache**. `ResourceManager` holds `unordered_maps` of textures, fonts and sound buffers keyed by path and hands out references; `SpriteSheet` holds a non-owning `const sf::Texture*` into that cache plus a map of named frame rectangles. Sharing a texture between a hundred Goombas is what any sane resource loader does; there is no intrinsic/extrinsic split and no flyweight object. Calling it a cache is the accurate description, and one fewer claimed pattern is a better report than one more false one.

Also confirmed absent, checked rather than assumed: Prototype, Builder, Null Object, Facade, Chain of Responsibility, a custom Iterator, Abstract Factory and Bridge. None appears in the codebase, and none is claimed.

8 Implementation details

8.1 The frame

A fixed-timestep accumulator: input is drained from the OS queue, `update(1/60)` runs as many times as the accumulator owes, and rendering interpolates between the last two states. Physics is therefore deterministic and independent of frame rate — which is what makes the Memento rewind (§8.7) reproducible, and what lets the CPU opponent be judged against the same simulation a human plays.

```
Game::run()
poll OS events ..... InputManager records held keys from the event STREAM,
not from sf::Keyboard::isKeyPressed - see 8.3
accumulator += frameTime
while accumulator >= 1/60:
  PlayingState::update(1/60)
  0.  rewind check (R held) -> restore a Memento and return
  1.  held keys -> command objects -> Player
  2.  AIController decides, for a CPU opponent
  2c. Shadow Mario samples this frame's inputs
  3.  for each entity: update()          <-- may REQUEST spawns
  4.  PhysicsEngine::update(entities, tilemap)    see 8.2
  5.  flushPendingSpawns()                <-- only point the list may grow
  6.  prune inactive; recycle pooled types into their ObjectPool
  7.  hazards: lava underfoot, the void plane, warp pipes, boss arena
  8.  P-Switch clock, HUD sync, boss HUD
  9.  record one Memento
  accumulator -= 1/60
  render, interpolated
```

Step 5's position is not cosmetic; §10.6 is the crash that put it there.

8.2 The physics pipeline

`PhysicsEngine::update` runs ten ordered steps — nine numbered stages in the code, with the broad and narrow phases sharing one stage and split into two rows below for readability — and the order is load-bearing at almost every step.

Stage	What it does, and why it is here
1. Conveyor push	Applied using the <i>previous</i> frame's ground status, because this frame's has not been computed yet and a conveyor must not act on someone who has already left it.
1.1 Interactive tiles	Coin tiles are not solid, so the collision detector never reports them. The player's footprint is scanned separately — see §10.5.
1.5 Acceleration and friction	Per-character: Luigi accelerates to $0.85\times$ the speed and keeps more momentum. Ice reduces friction; this is where surface type enters.
2. Gravity	Scaled by <code>getGravityMultiplier()</code> , so blocks (0) and projectiles (0) are skipped, water is $0.3\times$, and Luigi falls at $0.9\times$. Clamped at <code>TERMINAL_VELOCITY</code> = 600 px/s, or 60 in water.
3. Integrate X, resolve X	Only the maximum horizontal overlap is resolved. See §10.1: resolving every overlapping tile summed the push-outs.
4. Integrate Y, resolve Y	Separately from X, which is what makes walking into a wall while jumping behave correctly. Brick destruction from below happens here.
4.5 Map bounds	Everything is clamped to <code>[0, width × 32]</code> , so nothing walks off the side of the world.
5. Broad phase	<code>SpatialHash</code> with 64 px cells — two tiles, so a 32 px entity touches at most four buckets. Pairs are tested only against bucket neighbours.
6. Narrow phase and resolution	AABB overlap, then <code>CollisionResolver</code> dispatches on the pair's types: stomp, damage, collect, bump, shell kick, boss.
7. Intent flags	Cleared last, once acceleration and resolution have consumed them.

Cost. The broad phase turns an $O(n^2)$ all-pairs test into $O(n)$ expected work against bucket neighbours; tile queries are a direct index into a dense array, so $O(1)$ per probe. With a 200×23 map and a few dozen live entities the frame is dominated by rendering, not physics.

Feel. Two forgiveness windows, both six frames (100 ms): *coyote time* lets a jump register just after walking off a ledge, and the *jump buffer* lets a jump pressed just before landing fire on touchdown. Neither is physical; both are what stops the game feeling stiff.

8.3 Input: commands, and why not `isKeyPressed`

Each action is an `ICommand` object, so bindings are data: `InputManager` holds a per-player action→key table loaded from `config.json`, and Player 2 is the same commands through a second table. Rebinding is a map write.

Held state comes from the *event stream*, not `sf::Keyboard::isKeyPressed`. That is a deliberate correction: `isKeyPressed` reads global OS state, which on macOS returns false without Input Monitoring permission — so movement and the rewind key were dead for an invisible, machine-specific reason. Reading the stream also means losing window focus releases everything, rather than leaving a key stuck down.

8.4 Entities: one door in, one door out

Everything enters the world through `EntityFactory::create()` and then `admitEntity()`, which wires its animations and applies the difficulty and New Game+ modifiers. Because that is the single door, a difficulty change reaches an entity that does not know difficulty exists.

Leaving is symmetric: the prune step uses `std::stable_partition` rather than `remove_if` — `remove_if` leaves the tail moved-from, so the dead objects would already be gone before they could be offered back to a pool. Partitioning permutes instead, and the tail is real objects. `forgetEntity()` then clears every raw pointer the state holds into the list (the active boss, the shadow, Player 2, a carried shell) *before* the `unique_ptr` behind it is released.

`ObjectPool<T>` trades in `unique_ptr<T>` rather than owning its objects, so a pooled fireball is stored exactly like an unpooled Goomba and the entity list never learns that pooling exists. The pool caps what it retains: an unbounded free list is a leak that never frees.

8.5 Enemy behaviour and the boss template

An enemy holds an `IMovementStrategy*` and delegates to it. Eight exist — patrol (turns at ledges and hazards), chase, fly, tethered chase (Chain Chomp), hammer throw, timer emergence (Piranha Plant), linear (Bullet Bill) and proximity trigger (Thwomp).

A boss is not a strategy, because a strategy describes one repeatable motion and a fight is a sequence whose behaviour depends on its own health. `Boss::update()` is `final` and sequences the shared parts — i-frame countdown, stagger clock, phase transition, defeat animation — calling the pure-virtual `updateBehaviour()` for the subclass's attack pattern. Bowser adds fire breath on a phase-dependent interval, a phase-2 leap, and the fireball-stagger window; BoomBoom adds a charge.

8.6 The tile map and level loading

A dense `vector<TileType>` of 11 types, indexed directly. Level JSON stores tile *spans* (`{"type": "ground", "x": 0, "y": 21, "w": 170}`) rather than cells, so a 200×23 level is a few dozen lines. Themes select the atlas frames and the parallax backdrop.

Loading resolves its path against several candidate roots so the game runs from either the source tree or `build/`. That fallback had a real bug worth remembering: a single `ifstream` was reused across candidates, and the first miss set `failbit`, so every later `open()` was silently ignored (§10.9). After load, three passes run over the finished level: the backdrop's ground line is derived from the widest solid row in the lower half of the map; the flagpole and castle are settled onto whatever floor is beneath them; and the void plane is set one tile below the deepest floor.

8.7 Time rewind, replay and the snapshot

`GameSnapshot` holds both players' stats, the level timer, the camera centre, and one record per entity. Holding `R` pops snapshots off a 300-frame deque — five seconds at 60 Hz — and applies them.

Two details make it correct rather than approximately correct. Entities are restored **by id**, not by index: pruning and spawning permute the vector between capture and restore, so index restoration assigned positions to the wrong entities. And applying a snapshot now *cancels an in-progress death*, because restoring a position while leaving the player in its dying state put them back above the pit still falling.

`ReplayRecorder` is the same snapshot stream kept longer and thinned, capped at 6000 frames — roughly ten minutes — because a recorder with no cap is a memory leak with a feature name.

8.8 The CPU opponent

`AIController` drives a real `Player` through the same command objects a human uses. It has no privileged access to the world, only a wider view of it: each frame it builds an `AIObservation` — a 21×15 tile grid centred on itself, each cell classified *Unknown* / *Empty* / *Solid* / *Hazard* / *Reward* / *Enemy*, plus normalised offsets to its objective and to the opponent, its own velocity, and three flags.

Unknown is a distinct state from **Empty** on purpose: difficulty is expressed as vision radius, so an Easy bot genuinely cannot see a pit it has not reached rather than seeing one and pretending otherwise. Difficulty also sets reaction latency and action noise; the archetype (Speedrunner, Hunter, Collector) sets what the objective *is*. The policy sits behind an **IAIPolicy** interface, so the heuristic implementation can be swapped without the controller changing.

8.9 Multiplayer

Four modes on one shared screen. The camera frames the *midpoint* of both players and a tether keeps them together — hard in Versus, where whoever falls behind is shoved along at the edge, and soft in Co-op, where the leader is blocked at the right edge instead so the screen simply waits. Split screen was rejected: every screen-space overlay in the game would have had to learn about viewports.

Shadow Chase is the odd one. **ShadowMario** is a **Player** replaying the human’s own input stream three seconds late, with a correction lerp to stop it desyncing. It is a contact hazard and is unmoved by collision, because shoving it off its path would desync it from the recording driving it.

8.10 Graphics

Sprites come from JSON-described atlases (`player.json` alone has 236 frames — 59 per playable character); **Animator** plays named clips against them. The camera does look-ahead in the direction of travel, clamps to level bounds, and applies screen shake *between two clamps* so a shake cannot push the view outside the map. The parallax backdrop composes per-theme layers at fractional scroll rates, placing decorations by a hash of world position so they do not shimmer as the camera passes. Particles run on an object pool with a fixed capacity, so a fifty-coin burst allocates nothing.

8.11 The entity catalogue

One table maps every **EntityType** to its serialised name, display label and palette category. The JSON parser and the editor’s palette both read it, and a CTest case asserts that every entry is constructible by the factory and that each class’s own `getTypeName()` matches its catalogue name — because a mismatch means a level saved from the editor loads back as a different entity. Before this existed the list was hand-written in three places and had drifted: §10.4.

8.12 Procedural generation, solvability and Endless Mode

MapGenerator builds a level from a seed: multi-tier elevation, biome-specific ceilings, lava pits, reachability-guarded platform gaps and a threat-pacing curve. It drives three player-facing modes — the date-seeded Daily Challenge (deterministic for every player on the same day), a "Generate & Play/Edit" page, and Endless Mode.

Endless Mode is a true infinite runner, not a wide level. The tilemap starts as one generated chunk; `PlayingState::extendEndlessLevelIfNeeded()`, called once per frame, appends a fresh 100-tile chunk whenever the player nears the current edge. Each chunk is generated into an *isolated* tilemap and entity list — the generator unconditionally clears whatever entity vector it is handed, which would otherwise delete the live player — then spliced in at an x-offset with a flat safety bridge across the seam. Pit probability, enemy rate, terrain roughness and the difficulty tier all rise with the chunk index; no flagpole ever exists, and distance travelled is the score, recorded distinctly from the other modes on the Game Over screen.

Solvability. Every generated level and every Endless chunk is checked by `LevelSolvability`, an independent column-reachability BFS bounded by the game's own jump and run constants from `Constants.hpp` (a moving or falling platform counts as standable ground at its column, so a platform-bridged pit is not a false rejection). `MapGenerator::generateSolvable()` retries with a new seed on failure, bounded; if every attempt fails it keeps the last layout and logs the failure rather than blocking play — a check with retries, not a shipping gate (§13). The idea was salvaged from an abandoned GAN/RL side branch whose quality gate outlived its framework; the shipped implementation is a dependency-free rewrite, stress-tested by a CTest case across every theme and difficulty the menu can produce.

9 Data storage and serialization

Everything persistent is JSON via `nlohmann/json`, chosen over a binary format because a level file that a human can read and hand-edit is worth more during development than a few kilobytes.

File	Holds
<code>assets/levels/*.json</code>	Tile spans (run-length encoded by <code>w</code>), entity placements with per-instance fields such as Bowser's arena, spawn point, theme, dimensions.

File	Holds
<code>assets/config/entities.json</code>	Per-type tuning — speed, score, health — applied by the factory after construction, so balance changes need no rebuild.
<code>saves/slot_N.json</code>	Three save slots: form, lives, coins, score, position, star coins.
<code>saves/progress.json</code>	Campaign completion and unlocks; drives the world map and the New Game+ counter.
<code>saves/config.json</code> , <code>highscores.json</code>	Key bindings, volumes; the high-score table.

Tile-name and entity-name conversion each exist in exactly one function. That is a rule with a scar behind it: a duplicated string-to-enum chain in the level loader once drifted from the canonical one and silently dropped every coin tile in all seven level files.

10 Technical problems and solutions

Grouped by what they have in common rather than by the week they happened, because the groupings are the lesson. Each is drawn from the weekly reports (`docs/Group52_04/–10/`) and the session log, where the diagnosis was written down at the time.

10.1 Collision: the ground was not solid

Five separate defects, all in the first month, all producing "the player behaves oddly on flat floor". They are worth listing together because each looks like a physics-tuning problem and none of them was.

Symptom	Actual cause	Fix
The player sticks at random points while walking across flat ground	A horizontal overlap at the seam between two floor tiles was being read as a side-wall collision, so <code>velocity.x</code> was cancelled and <code>onWall</code> set.	<code>CollisionDetector</code> now looks at the tile <i>above</i> the one it hit: if that is empty, this is a floor seam, not a wall, and the push-out is converted to vertical.

Symptom	Actual cause	Fix
<code>onGround</code> flickers true/false every frame, and the player jitters	<code>resolveEntityVsTile</code> moved <code>position</code> but not <code>boundingBox</code> . The stale box was still inside the tile next frame, re-triggering the overlap.	The bounding box is synchronised at the point of displacement.
Standing across two tiles launches the player slightly into the air	Both tile collisions resolved in the same frame and the two push-outs <i>summed</i> .	Resolve only the collision with the maximum penetration per axis per frame. That is sufficient: clearing the deepest overlap on a flat surface clears the shallower ones with it, so one correction does the work of all of them without double-counting.
Holding a direction against a wall in mid-air lets the player glide up it	Nothing removed vertical momentum on a mid-air wall contact.	Wall friction: upward velocity is zeroed on an airborne horizontal hit, and falling is capped at <code>WALL_SLIDE_SPEED = 50 px/s</code> — which is also where the wall-slide mechanic came from.
Mario cannot enter pipes or drop through one-tile shafts	His hitbox was the full 32 px tile width, so every one-tile gap was an exact fit and caught on the corner AABBs either side.	Hitboxes narrowed and centred inside the sprite: 24×30 Small, 24×60 Super, 14×14 Mini. A 4 px margin each side is the whole fix.

The pattern: four of the five were a *bookkeeping* error — state updated in one place and not the matching one — rather than wrong physics. The fifth was a geometry assumption nobody had written down.

10.2 Gravity applied to things that should not fall

`Block` derives from `Entity`, and `Entity`'s `getGravityMultiplier()` returns 1. `Block` did not override it, so the physics engine accelerated every question block and brick downwards at 1800 px/s² and the level geometry fell out of the bottom of the map. The fix is one line — `return 0.0f` — but the interesting part is why it was not caught: the blocks were correct in the level file, correct on the first rendered frame, and wrong a second later.

The same defect in a different costume: a mushroom dispensed from a block had no resting state, so `resolveEntityVsTile` corrected its penetration without zeroing its accumulated downward velocity,

and it eventually slipped through a seam and fell out of the world. `Item` gained an `m_onGround` flag and gravity now skips a resting item.

10.3 Object lifetime, four times

The most expensive category in the project, and the one whose members look least alike.

- **Dangling event subscriptions.** `PlayingState` subscribed to `PlayerShotFireball` and never unsubscribed, so an event published after the state was destroyed called into freed memory. Every subscription in the state now has a matching `unsubscribe()` in `exit()`.
- **An invalid-handle sentinel that was a valid index.** Subscription IDs were compared as raw integers, so an "unset" handle could address a real subscriber. The sentinel is `static_cast<size_t>(-1)`.
- **Shutdown order.** Closing the window destroyed the singleton managers while game states were still alive, and the state destructors called into them — a mutex failure on the way out. `Game::shutdown()` now pops every state *before* shutting the managers down, and closes the window before that, because SFML's OpenGL context was being torn down while the render window still held handles to it (an immediate `SIGABRT` at exit).
- **The mid-frame spawn crash** — §10.6, the culmination of this theme and the only one that was platform-dependent.

10.4 One fact, two places

The project's signature failure. In each case nothing failed to compile; the program simply did something different from what the code appeared to say.

The duplicated fact	What it cost
The tile name $\leftrightarrow$ <code>TileType</code> mapping, re-implemented in <code>LevelLoader</code> alongside the canonical one in <code>SerializationUtils</code>	The copy drifted and silently dropped every coin tile in all seven level files . Both directions now live in one pair of functions, and a round-trip test guards them.

The duplicated fact	What it cost
The entity type list, hand-written in the factory, the parser and the editor palette	The editor could not place a Goomba, a Koopa, a pipe or a flagpole — 16 of ~40 types, no enemies at all. Now one <code>EntityCatalogue</code> with a parity test (§8.3).
The flagpole's height: 168 px of sprite against a 300 px collision box	The pole you could touch and the pole you could see were different objects, and the catch-height score was measured against one that did not exist.
The backdrop's ground line: a hardcoded screen constant against the real tile geometry	§10.7 — it took three attempts.
The class diagram, hand-maintained against the headers	Rotted until it omitted most of the architecture while still reading as authoritative. Now generated (§6).

The standing rule this produced: the commit that creates the second copy also creates the test that fails when the copies disagree. Writing this report exercised it — the build script counts what it claims, and caught a class count inflated by a substring match and "23 test files" being reported as "23 things CI runs".

10.5 Interaction that was never wired

Several features existed as far as the event that announced them and no further.

- **Coins on the tile grid could not be collected.** `TileType::Coin` is not solid, and the collision detector only reports solid tiles — so walking through one did nothing and the coin stayed on the grid forever. The physics engine now scans the player's footprint for non-solid *interactive* tiles after integration.
- **The coin counter ignored its own payload.** `AchievementManager` did `m_coinsThisRun++` on `CoinCollected` rather than reading the amount, so a block dispensing ten coins counted as one.
- **The P-Switch and POW block** published an event that reached a sound and nothing else — no brick/coin swap, no enemies flipped, and the HUD's countdown field was never set.
- **Six subsystems were complete and inert:** minimap, particle system, death effects and screen transitions all compiled, all had passing harnesses, and none were ever constructed by the game.

***The rule, not the patch.** A task is complete only when the code is reachable from `main()` and has been observed running. A harness proves a class works in isolation; it does not prove the game ever builds one. That is why §4 records how each capability was confirmed. The menu music, which had been failing silently at startup for weeks, was found by running the game for six seconds — not by any of the test harnesses.*

10.6 A crash on Windows that could not be reproduced on macOS

A teammate reported that the game crashed in 1-3, and that deleting Bowser from the level file made it stop. The same build played 1-3 correctly on macOS.

The cause was not in Bowser. Every spawn travels through a synchronous event: an entity that wants to create another publishes a request, and `PlayingState` performs the spawn in a subscriber. That subscriber called `m_entities.push_back()` — on the stack of the `for (auto& entity : m_entities)` loop that was calling the publisher. A range-for caches `begin()` and `end()` once, so when a `push_back` exceeded capacity and reallocated, the loop's iterators pointed into freed memory.

That is undefined behaviour, and undefined behaviour is allowed to work: the macOS allocator left the freed page mapped and readable while the Windows toolchain faulted. Bowser mattered only because he is the one entity that spawns another several times per second, so 1-3 reaches a reallocation almost immediately.

***Solution.** Spawns queue and are admitted at exactly one point per frame, after both the update loop and the physics pass have finished. Full analysis, with the amortised-growth argument and a worked trace, in [docs/learning/mid-frame-entity-spawn-crash.html](#).*

10.7 A boss that could not be beaten, and a backdrop that took three tries

Bowser was near-impossible for a structural reason rather than a numeric one: he is immune to fire, a boss carries a second of invulnerability after every hit during which contact *damages the player*, and he advances breathing fire throughout. The only legal input was five clean descending stomps

with no way to create an opening. Two routes were added, both from the series — four fireballs now stagger him into a three-second window where a stomp is safe and lands, and the level ends on a bridge over lava with an axe beyond it that drops both.

The backdrop is the better cautionary tale. The parallax layers drew on a hardcoded screen line of 640 px; the real ground renders at 656. The *first* fix anchored them to the lowest solid row — which is the bottom of a two-row floor slab, so they were buried a tile instead of floating one. The *second* correctly found the top of the slab, but the renderer still decided ground-versus-sky by comparing a layer's authored baseline against the runtime ground line, so every ground layer was reclassified as sky and pinned to the stale constant anyway. Only the third fix — making "this layer stands on the ground" a property of the layer, and restricting the ground scan to the lower half of the map so a castle ceiling cannot win — was correct. It was confirmed by measuring pixel columns in captured frames, not by reading the code again.

10.8 Process failures, and the rules they produced

Three of the most costly problems were not in the code.

A destructive git command deleted a week's work. A session ran `git reset --hard` ~~ES~~ `git clean -fd` to unblock a build, permanently destroying untracked files including the entire Week 8 report. Rule: never discard uncommitted work to unblock a git operation — committing is reversible, discarding is not — and record the before/after commit hashes of every version-control operation.

An audit was published from a stale checkout and had to be retracted. Its first revision was written against a local `dev` nine commits behind, and on that basis declared the entire audio system missing. The audio system was implemented and merged. Rule: fetch before describing repository state; a local branch is not evidence.

~3,100 lines of finished work were sitting uncommitted — three sub-level maps, the tools directory and a week's report — one careless clean from being lost. Found and committed during the audit.

A fourth, found while writing this report: the test suite read and wrote the developer's real `saves/` directory, and a `ctest` run was observed deleting actual campaign progress. One high-score

assertion also passed under `ctest` and failed running the same binary from `build/`, because the two have different working directories. Fixed at the root — every harness now redirects `save` to a temporary directory, and a test asserts that containment.

10.9 Toolchain and porting

Smaller, but each cost real time.

- **SFML 3.0.2 removed `sf::Vertex`'s constructors.** Parenthesis construction no longer compiles against an aggregate; brace initialisation satisfies both 2.x and 3.x.
- **A stream that stayed failed.** `LevelLoader` tried candidate paths in a loop with one `ifstream`. The first miss set `failbit`, and every subsequent `open()` was silently ignored — so sub-levels failed to load depending only on which directory the game was launched from. One `file.clear()` per iteration.
- **Non-uniform randomness.** `rand() % range` for particle spread angles clustered visibly; replaced with `std::uniform_real_distribution` over `std::mt19937`.
- **Save files could contain nonsense.** Out-of-range player coordinates were written straight into the slot JSON and crashed on load; coordinates are clamped to the level bounds before serialisation.
- **Stale achievement toasts.** Loading a save did not clear the active toast queue, so previously-unlocked achievements popped up again after every load.

11 Features demonstration

All five figures are unretouched frames captured from the running game by a scripted input driver (`--script`), which synthesises events into the same path the OS event queue feeds. Nothing outside the process is touched, so a verification run is reproducible.



Figure 1 — The main menu, over the parallax backdrop. Eight entries including the four multiplayer modes, the map editor and the procedural generator page.



Figure 2 — World 1-3, Bowser's Castle. The HUD reads WORLD 1-3, taken from the level catalogue

rather than a constant. The castle towers and fencing of the parallax backdrop stand on the floor the player walks on, at the correct depth tint for the castle theme.

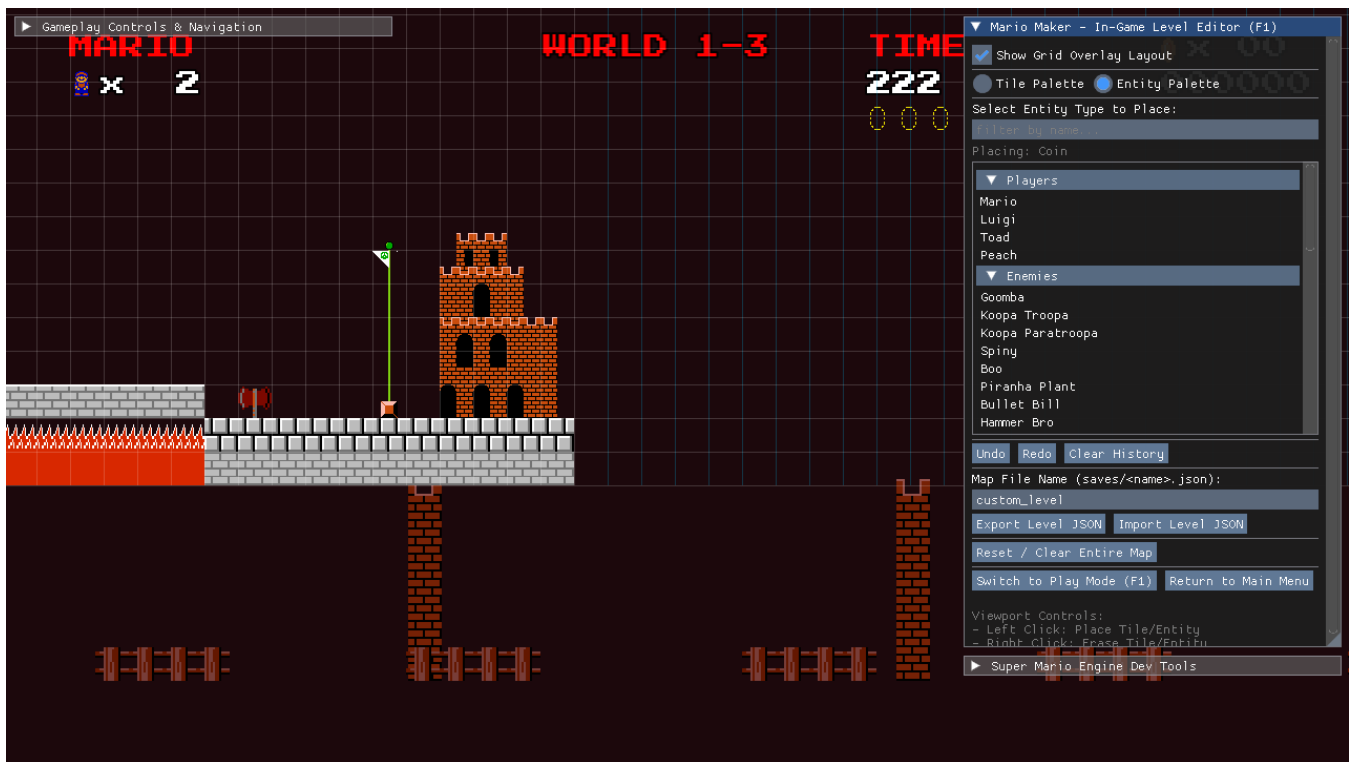


Figure 3 — The rebuilt end of 1-3, seen through the editor’s free camera. Left to right: lava spanned by the brick bridge Bowser paces, the axe that drops it, the goal flagpole standing on the ground, and the castle drawn from the atlas’s castle_end art. The editor’s tile palette is open on the right.



Figure 4 — Versus CPU. Both players have an icon and a life count in the HUD; the bottom strip carries the score line and who is leading. The dev panel reports the opponent's policy and what it currently believes it is doing ("crossing gap") — the CPU drives a real Player through the same commands a human uses.

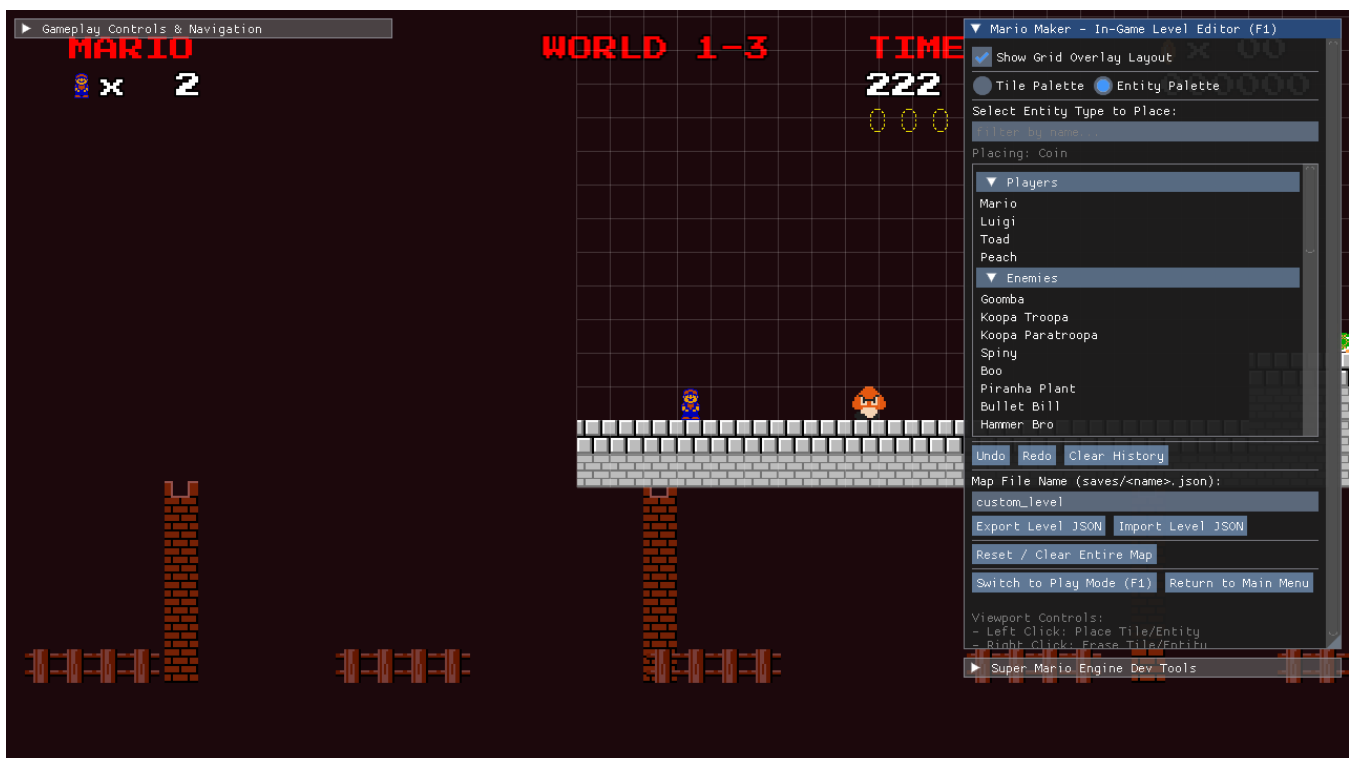


Figure 5 — The in-game level editor (F1). The entity palette is generated from the single catalogue and grouped into Players, Enemies, Items, Blocks and Scenery, with a filter box and the serialised name on hover. Edits go through Command objects, so undo and redo come for free.

12 Process and project history

The project ran from 29 May to 2 September 2026 across 10 weekly reporting periods (docs/Group52_04/ through Group52_13/), 513 commits and 302 recorded working sessions in logs/agent_history.log. This section is about the third of those numbers rather than the first two: **how the project checked itself, what the checking produced, and why its engineering rules read the way they do.** The individual defects are deliberately not restated here — the five worth reading are worked through in §10, and the full ledgers live in the audit documents named below.

12.1 Timeline

Period	Dates	What landed
W04	29 May – 4 Jul	Environment and architecture. Game loop and <code>GameStateManager</code> , collision primitives and spatial hash, math utilities, the entity skeletons. Member B started the input and sound spine.
W05	5 – 11 Jul	Physics made real: camera, momentum and friction centralised into <code>PhysicsEngine</code> , wall sliding, and the JSON <code>LevelLoader</code> . Member B’s gameplay entities, blocks and the animation system.
W06	12 – 18 Jul	Persistence and tooling: save slots, statistics, achievements, the pause and settings panel, and the first version of the level editor with its undo/redo Command framework.
W07	19 – 25 Jul	First real integration of both members’ branches, the consolidated <code>verify_all</code> runner, the entity factory pipeline, and the design for two-player and Shadow Mario.
W08	26 Jul – 1 Aug	Particles on an object pool; repository synchronisation after the branches had diverged.
W09	2 – 8 Aug	Fireballs through the pool, flagpole scoring, trampolines, checkpoints, the Memento time-rewind, and the first procedural generator.

Period	Dates	What landed
W10	9 – 15 Aug	The three-world campaign with warp-pipe sub-levels, physics normalisation for static bodies, and hitbox tuning.
W11	16 – 22 Aug	The first checkpoint audit and its remediation, the first green CI, the boss fights, multiplayer and the CPU opponent, and the Windows crash root-cause in §10.1. The heaviest week of the project.
W12	23 – 29 Aug	One commit, on every branch. Reported as a quiet week rather than padded with the adjacent weeks' work.
W13	30 Aug – 5 Sep <i>reported to 2 Sep</i>	The specification audit, its remediation batches, and the submission sweep: pipe entry modes, save-slot selection, the lighting renderer, the level editor's custom levels, the entity registry, and the documents this report is part of.

12.2 The procedure: checkpoint audits, repeated

The project's verification is not one final review. **Four times, work stopped and the tree was audited against something outside itself** — the specification, the reports' own claims, or a reader who had written none of it. Each audit produced a numbered defect ledger; each ledger was worked off in batches; each batch left a test behind.

18 August — code audit. A full read of both members' domains against the code reachable from `main()`. It produced **37 findings, 7 of them critical** (GitHub issue #11), and it is the hinge of the project: three of the rules in `AGENTS.md` exist because of what it found, not because of what it fixed (§12.4).

31 August — specification and claim audit. Three parallel read-only passes reconciled `SPEC.md` v2.0's frozen 110-feature specification against the code, and — the part that matters for a report — against *every claim the project's own documents made*: the features list, this report's prose, both task checklists. Twenty-eight defects were raised. On re-verification **four were struck**: one refuted by live evidence in the same session that recorded it, one describing an animation that does not exist in the codebase, one already fixed twelve days earlier, and one whose stated mechanism is absent from the code (§13). The audit recorded its own methodological failure — observations had been folded in without re-verification against the commit being audited — and made re-verification-before-recording a standing requirement.

2 September — submission sweep. The audit this report was written inside. It re-derived the

tree's facts from the tree, rebuilt every stale document, and ran five defect lanes in isolated worktrees off one frozen baseline. Its own execution record is `docs/issues/submission_sweep_plan_2026-09-02.md` §8, written as the work happened rather than afterwards.

The sweep's own per-lane review round. The fourth audit is the one inside the third. No lane's report was accepted on its word: **the reviewer re-ran the lane's own measurement** — its `cctest` run, its commit range count, its build — before accepting the branch. Several lanes were sent back for rework, one of them three times over counts it had typed but could not measure. Several others came back having *corrected the plan that briefed them*: one lane was told to archive a document that turned out to be cited by section number from seven live engine sources, and refused; another was told to fix a defect that did not exist, and proved it (§12.4). That is the outcome a review round exists to produce.

Remediation in numbered batches. Every defect got a number and its own small project: a branch, a fix, a guard that fails without the fix, and a log entry recording what was actually run. The two audits' remediation ran as numbered batches **R1 through R21** (one of which, R18, was cancelled after the defect it targeted was struck). The numbering is the useful part — a defect that is cited by number in a commit message, a test name (`verify_r21_flagpole_softlock`), a code comment and a log entry cannot quietly become "probably fixed".

A guard does not count until it has been watched failing. Every new test is mutation-tested: the fix is reverted, the guard is observed to *fail*, the fix is restored, and the guard is observed to pass again. This catches the failure mode a green suite cannot — a test that passes for the wrong reason. The clearest instance in the sweep: a guard for a **Spiny** that must hatch from an egg before it walks. The obvious mutation, reverting the constructor default, would have left the guard **passing vacuously**, because a Spiny that was never an egg satisfies "ends up walking" from its first frame. The lane recognised that and mutated the hatch trigger itself instead. Honesty runs the other way too: of one lane's seven mutation attempts, **one survived** — the estimate it targeted already floored to the same value — and it is recorded as a surviving mutation in the log rather than quietly replaced with a luckier one.

"Complete" means reachable from `main()` and observed running. Not that a file exists, compiles, and has a passing harness. This is the project's most expensive lesson priced in advance: the 18 August audit found **six subsystems that were "complete" and inert** — compiled, harnessed, green, and constructed by nothing the game ever reached. Every claim in §4 of this

report is graded against the stronger definition, and where only the weaker one holds, §13 says so.

The log is the audit trail. `logs/agent_history.log` holds 302 entries, and it is not a changelog. Each records the commit before and after, whether the remotes were fetched, whether the work is reachable from `main()`, how it was verified, and — required, not optional — the mistakes made getting there. It is append-only and **union-merged**: when two branches both appended, the resolution is to keep both entries, never to pick a side, because a deletion destroys evidence while a duplicate merely wastes a line. Several passages of this report, including §12.4, are drawn from it.

CI is the part that does not depend on anyone remembering. GitHub Actions builds the game and runs the registered test suite on every push to the integration branches (`.github/workflows/ci.yml`), inside a virtual framebuffer and against a dummy audio device so that a harness needing a display or a sound card still runs on a headless runner. A rule only a human applies is not enforced; CI is what makes "reachable and observed" checkable by something other than good intentions.

12.3 What the audits produced

Mechanism	What it produced	Standing effect
Code audit 18 Aug	37 findings, 7 critical. Six subsystems complete and inert. About 3,100 lines of finished work sitting uncommitted. The audit's own first revision retracted.	Three <code>AGENTS.md</code> rules, each naming its incident. The first green CI.
Specification and claim audit 31 Aug	110 spec features reconciled against code and against every document claim. 28 defects raised, 4 struck on re-verification.	Batches R1–R20. An observation must be re-verified against the audited commit before it is recorded.
Submission sweep 2 Sep	Five defect lanes off one frozen baseline; every stale document rebuilt from the tree; one claimed defect withdrawn; a defect found in this report's own fact extractor.	Counts in the generated documents come from the tree, never from prose. The sweep's execution record is itself a document.
Per-lane review round within the sweep	Every lane's measurement re-run rather than trusted. Several lanes reworked; several came back having corrected the plan that briefed them.	A lane with no way to measure a number will still write one down. Review re-measures.

Mechanism	What it produced	Standing effect
Mutation testing	Every guard observed failing with its fix reverted. One vacuous guard caught before it shipped; one surviving mutation recorded rather than dropped.	A green test is evidence only once it has been seen to go red.
Reachable-and-observed rule	Six inert subsystems wired in one session; the menu music, silently failing for weeks, found by running the game for six seconds.	§4 grades every capability by how it was confirmed; §13 lists what only compiles.
The log	302 entries carrying fingerprints, fetch status, reachability, verification and mistakes.	Union on merge conflict — keep both entries, never choose a side.
CI	Build plus the registered suite on every push to the integration branches.	The rules stop depending on whoever is at the keyboard.

12.4 The rules came out of the failures

This is the part of the process worth defending, because none of the project’s engineering rules were adopted on principle. Each is a scar, and `AGENTS.md` records the incident beside the rule so a later reader can judge whether it still applies.

A `git reset --hard` ~~to~~ `git clean -fd` destroyed a week’s progress report. It was run to unblock a git operation. The Week 8 report in `docs/Group52_08/` was uncommitted, and it did not come back. A later session then found about 3,100 lines of finished work — three sub-level maps, the tools directory, a week’s report — one careless clean from the same fate. Rule: never discard uncommitted work to unblock a git operation; commit it, because committing is reversible and discarding is not. Only `imgui.ini`, which the UI regenerates, is exempt.

An audit was written against a stale local branch and had to be retracted in public. The 18 August audit’s first revision was written against a local `dev` nine commits behind, and on that basis declared the entire audio system missing. The audio system was implemented and merged on `origin/dev`. The finding was withdrawn on issue #11. Rule: `git fetch --all` before any task whose output describes repository state, and record in the log that it was done. Every audit and weekly report in this project since carries that line.

*The same defect kept arriving in different clothes. Three of the five problems in §10 are one failure: **a fact stored in two places, drifting apart, with nothing comparing them.** The entity list in the factory, the parser and the editor palette. The tile name-to-enum mapping duplicated in the loader, which silently dropped every coin tile in all seven level files. The flagpole's collision height against its sprite height. None produced a compiler error; all produced wrong behaviour that looked like a design choice. Rule: the commit that creates the second copy also creates the test that fails when the copies disagree. The entity catalogue's registry walk (§8) is the model.*

The last of those rules caught this report. The document you are reading is generated by a script that counts what it claims, and the counting has already found two wrong figures before they were printed — a class count inflated by a substring match, and "23 test files" reported as if it meant "23 things CI runs". The sweep found a third: the extractor that counts registered test cases could not see the guards registered through a second CMake mechanism, so the report was *understating* its own verification. A number that is computed can still be computed wrongly; what a parity check buys is that the error surfaces as a failure rather than as prose.

12.4.1 The audit corrects itself

The strongest evidence that this procedure works is not a defect it fixed but a defect it **withdrew**. The submission sweep's own plan asserted that the third axe in the Hard Bowser fight stood behind the arena enclosure, unreachable by ordinary movement, making that fight unwinnable — a critical, plausible, level-data defect with an obvious remedy. Told to fix it, the lane instead **tried to reproduce it**: it confirmed in code that the Hard difficulty really does require all three axes, then drove a real `Player` through real `PlayingState::update()` frames and reached the axe cleanly on the unmodified level file. It also tried the remedy the plan proposed — removing the enclosure's solid tile — and found it changed nothing. **No level data was edited.** The claim was withdrawn, and a mutation-tested reachability guard was left in its place, one that fails if the axe is ever moved outside the arena's boundary. A procedure that only ever confirms its own findings is not an audit; this one is allowed to say it was wrong, and did.

13 Verification, CI and known gaps

The tree holds 40 verification harnesses; 38 of them are built by CMake, and the 32 registered as CTest cases are run by GitHub Actions on every push to the integration branches. Not every harness is registered: those that open a window cannot run on a headless runner, so they are compiled but not executed there — "number of test files" and "number of things CI runs" are different figures, and reporting the first as the second would overstate what is verified automatically. Alongside the harnesses, CMake registers the standalone hygiene guards `guard_saves_hermeticity` and `guard_asset_single_source`, which check the repository rather than the gameplay. Cases are added whenever a defect escapes to an audit, so the suite is a record of every bug the project has shipped rather than a plan someone wrote in advance.

The suite is hermetic (R11, docs/issues/spec_feature_audit_2026-08-31.md): every harness's `main()` opens a `TestSaveSandbox` (`tests/TestSaveSandbox.hpp`) that points `Serializer::setSaveDirectory()` at a throwaway per-process temp directory before any other code runs, so nothing under test can reach the developer's real `saves/`. That the seam exists is not proof every harness uses it correctly, so `guard_saves_hermeticity` (`tests/guard_saves_hermeticity.cpp`) checks it empirically: a CTest fixture snapshots the content of the real `saves/` directory before any `verify_*` case runs and re-snapshots it after every one has finished, failing the run if a single byte differs or a file was added or removed. Verified by mutation, not merely by a green run: disabling one harness's sandbox line reproduced the original incident — the guard caught both a rewritten `config.json` and a newly created `profile.json` — and re-enabling it turned the guard green again on the next run.

13.1 Known gaps

What follows is the honest state of the project on the day it was submitted, grouped by the kind of gap rather than by severity. It is written to the project's own definition of complete (§12.2): reasoning about code is not observation of it, so anything verified only by reasoning is listed here even where the reasoning is sound.

13.1.1 Claims this project cannot verify with the environments it has

- **No 3D renderer.** The rubric's 5-point line is forfeited, deliberately and with the trade-off argued in §4.
- **The Windows platform claim is unconfirmed, and will stay that way.** CI runs on `ubuntu-latest` only (`.github/workflows/ci.yml`), and no session in this project's history has had a Windows or MSVC environment. The crash's *fix* is independently verified: the undefined-behaviour analysis in `docs/learning/mid-frame-entity-spawn-crash.html` plus scripted runs that hold World 1-3 through Bowser's fireballs without a crash on macOS. What is unconfirmed is specifically the claim that a Windows build was produced and run — a log entry asserting one was struck as unsubstantiated during the 31 August audit, because it modified no source and no build configuration and there has never been Windows CI that could have produced it. The honest status is the earlier entry's: not confirmed on Windows.

13.1.2 A defect neither fixed nor dismissed

- **The death-sound complaint is not demonstrable as worded.** The playtest observation was that the death SFX is cut off by the respawn transition. Investigated: `lost_life.wav` runs 3.267 s, longer than any fall-plus-respawn window; no code stops it; the channel pool only reuses channels that have already stopped; and it plays on over the resumed BGM. So the stated mechanism is absent from the code, while the observation itself came from someone who was listening. It is recorded as *neither fixed nor invalid*, and it needs someone with working audio to characterise what was actually heard. Quietly closing it as "cannot reproduce" would have been the dishonest option.

13.1.3 Verification that stops short of observation

- **Four level-placed Spiny hatches are unit-tested, not observed.** A Spiny now starts as an egg by default, which was necessary because every production construction path used the one-argument constructor and so produced a Spiny already walking in mid-air. That default also changes behaviour for four placements in the shipped campaign — three in `level_2.json`, one in `level_3.json` — and those four were *not* watched in a live run. They are covered by a mutation-tested post-condition (a Spiny built the way `LevelLoader` builds one must reach the walking state

once grounded, which asserts the outcome rather than the default, so the test cannot license the bug it guards). Reading `Spiny::update()` shows an egg resting on ground hatches on its first grounded update — but that is reasoning, and this project’s rule is explicit that reasoning is not observation.

- **The Bowser stagger is observed but not tuned.** Four fireballs and a three-second window (`Bowser::FIRE_HITS_PER_STAGGER`, `STAGGER_SECONDS`) were watched firing live — the counter running 1 to 4, the HUD switching to the stomp prompt, contact refused while he guards. What no run measures is whether those two numbers are *right* for a human player. They are reasoned and observed, not balanced.
- **Playtest coverage still lags harness coverage.** The 18 August audit named this imbalance and it has narrowed rather than closed: the log now records live scripted runs across every shipped world, driven by committed input scripts, but the harnesses still outnumber the recorded sessions by a wide margin. Every defect in §10.1, §10.4 and §10.5 was invisible to the test suite and obvious within seconds of watching the game run. The suite is not the problem — it is what makes the fixes stick — but it only verifies what someone already thought to doubt.
- **No sanitizer job in CI.** AddressSanitizer would have found §10.1 on its first run. The suite is now hermetic, which is the precondition for adding one; nothing else blocks it.

13.1.4 Performance left on the table, measured rather than guessed

- **PhysicsEngine::update() is not distance-gated.** It runs five loops over every entity, every frame, however far behind the camera that entity is. The off-camera freeze that Endless Mode needed landed on the game-state side of the update instead, and its own measurements are the reason this entry exists: at the third appended chunk, 23 of 90 entities were frozen while the deliberate exemptions accounted for 43 — a real but modest win whose dominant term is the exemption list, not the gate. `PhysicsEngine.cpp` sat outside that lane’s file budget and is the larger remaining cost. Deferred with the census counters left in place, so whoever takes it can measure the result instead of asserting one.

13.1.5 Design trade-offs accepted rather than fixed

These are decisions, not oversights, and §6 argues each one on its merits with the code that motivates it. They are repeated here because a known cost belongs in the list of known gaps.

- **PlayingState is a god class** — the largest file in the project by a wide margin, carrying level load, spawning, physics orchestration, camera, HUD sync, boss arenas, two-player, rewind, pipes, cheats, the editor bridge and endless chunk extension. Five subsystems have already been extracted from it; the remaining decomposition was not attempted in submission week, because a refactor of the file every other lane was editing is the wrong change to make under a deadline.
- **Twelve singletons, not four.** The specification claims four; the code has twelve reached from across the tree. §6 owns the real number and argues the construction-order rationale rather than restating the claim.
- **Friend sprawl.** `Entity` and `Character` each grant twelve friend declarations, giving the physics engine, the collision resolver, `PlayingState` and the movement strategies direct write access to protected state. Formally accepted as a deliberate trade-off in `SPEC.md` §22 with its rationale; the alternative was a mutator interface wide enough to be equivalent.
- **The `IPlayerState` axis is paid for and under-used.** Four of its five forms have empty bodies and the forms differ only by the size they report; only the cape form carries behaviour. Real per-form behaviour lives in the Decorator layer instead. Two axes were built; one of them does most of the work.
- **GameSnapshot is a fully public aggregate.** A C++ Memento written as a plain `struct` has no narrow interface, so the Originator's internals are readable anywhere the type is. A common simplification, and still a weakened encapsulation boundary.

13.1.6 Specification items that were not built

- **The descope list is explicit and dated.** `SPEC.md` §21 carries a "Descopes Addendum (2026-08-31)" naming every specified feature that will not be implemented, each with a reason: dynamic music layers, autoscroll sections, timed bonus rooms, climbing, swimming as a distinct state, the skid and hover-pause mechanics, knockback input-lock, red enemy variants, the cape swoop,

the Mini form's abilities, the character-switch hotkey, floating score text, extra object pools, A* pathfinding and split-screen speedrun. The addendum exists because the audit found the alternative in progress: features quietly missing from the code while the documents still claimed them. One entry has since been *un-descope*d — the GLSL lighting and weather line, because the shader was subsequently built for real (weather remains out of scope) — which is worth more than the list itself, since it shows the addendum is maintained rather than written once.

- **The solvability oracle records its own failure but does not gate on it.** If every bounded reseed attempt fails, `MapGenerator::generateSolvable` keeps the last unverified layout and returns `false`. All three call sites do consume that result: they set an `m_lastLevelUnverified` flag, warn on standard error, and the dev panel reports "layout unverified" in its generator section. What no caller does is *refuse to play the level*. "Checked with retries, and told when the check failed" is true; "verified before shipping" is not.

14 Conclusion and future work

The project delivers a complete, playable platformer whose value as coursework lies in its architecture: ten design patterns, each introduced because a concrete problem demanded it, over a four-level abstract hierarchy that the engine drives without ever asking what an object is. Adding a new enemy means one class and one catalogue row; adding a new level means a JSON file.

The more useful outcome is what the defects taught, and it is a single lesson repeated: a fact stored in two places drifts apart, and nothing notices unless something compares them. The standing answer — the commit that creates the second copy also creates the test that fails when the copies disagree — is now the project's most-used rule, and §12.4 traces it back to the three defects that bought it. The Windows crash taught the other half: "it works on my machine" is exactly the evidence undefined behaviour is best at manufacturing.

14.1 Future work: the far horizons

None of what follows is part of this submission. Two of the three exist as real branches that are deliberately kept off `dev`; the third is a direction with no branch at all. The game on `dev` ships the hand-authored campaign and the heuristic AI opponent,

and plays identically with every branch below dropped — which is the property that lets them be experiments rather than liabilities. Nothing here should be read as a feature that nearly landed. What is missing from this submission, as opposed to beyond it, is in §13.

14.1.1 Learned agents: a trained policy behind the seam that already exists

Not delivered — **branch A/r1-neural-policy, deliberately unmerged.**

The shipped game already draws the line a learning agent would need. `AIController` does the sensing and the actuating — it reads the tilemap and the entity list into an `AIObservation`, and it turns an `AIAction` back into calls on a real `Player`. What it does not do is decide. That sits behind one virtual function, `IAIPolicy::decide(const AIObservation&)`, returning an `AIAction` (`include/Entities/IAIPolicy.hpp`), with the shipped `HeuristicPolicy` as the baseline any trained policy would have to beat. Four properties of that seam were chosen for this horizon specifically, because each is cheap now and expensive to retrofit: the observation is **fixed-size and normalised**, so an easy opponent sees `Unknown` outside its vision radius rather than a smaller grid and one network accepts every difficulty; `AIAction` is **the same seven buttons a human presses** and that `PlayerFramePacket` already records for Shadow Mario, so a recorded human match is imitation-learning data without a conversion step; the exploration noise lives in the controller rather than the policy, so it applies identically to both implementations; and `decide()` is synchronous and allocation-free on the hot path.

What a trained policy would still need from the engine is the honest part of this horizon, and it is three things, none of them small:

- **Headless, accelerated stepping.** Training wants thousands of episodes; the game runs at 60 fps with a window open. This needs a loop that steps the simulation without rendering and without sleeping, and `PlayingState` assumes a render target exists in several places.
- **Determinism, or an explicit decision to live without it.** `ReplayRecorder`'s own header documents why the simulation is not reproducible from inputs alone: float physics, an entity list that spawns and prunes mid-frame, and strategies that read a shared singleton. Model-free learning survives that, because it only needs transitions — but anything that must replay a trajectory exactly is ruled out until the sources of divergence are removed one at a time.

- **A reward log.** The branch's `RewardTracker` shows the shape this should take: it subscribes to events the game *already publishes*, so no gameplay code knows a reward exists, and it writes one row per *decision* rather than per frame, because logging idle frames fills the file with duplicates of the same state.

The branch carries `NeuralPolicy`, `RewardTracker`, `ExperienceLog`, a verification harness and a design document (`docs/rl_training.md`). One detail there is worth repeating as engineering, whatever happens to the learning: the observation layout is treated as a *versioned contract*, and loading a weight file whose version does not match is refused rather than guessed, because a silently misaligned input layer produces a policy that acts confidently and arbitrarily — the hardest failure to diagnose from outside.

14.1.2 Learned level generation, with the game as the oracle

Not delivered — **branch A/mapgen-gan-plan, stacked on the RL branch and equally unmerged.**

The shipped `MapGenerator` is a single left-to-right pass of *independent* probability rolls: an elevation profile, a per-column pit roll, uniform decoration and enemy rates. That structure cannot express what makes a Mario level good, and the limitation is architectural rather than a matter of tuning: independent rolls produce no rhythm or phrasing, no difficulty curve across the level's length, and almost never a *composed* challenge where a pit and an enemy and a moving platform are one obstacle rather than three coincidences. A learned generator attacks exactly those three, because it learns tile *neighbourhoods* from levels a designer actually made. The direction is a DCGAN over the public VGLC *Super Mario Bros.* corpus, with levels authored in this project's own editor as a second corpus, generating chunks that are stitched, repaired and filtered rather than played raw.

The interesting half is the loop between the two horizons, and it has a name in the literature — agent-in-the-loop generation, and adversarial curriculum generation after it. Each side supplies what the other cannot: the generator gives an agent unlimited varied levels so it does not overfit three hand-made maps, and the agent gives the generator a **playability oracle**. A GAN learns what levels look like; only something that plays them knows what they play like. In this project's plan the cheap half of that oracle runs first, before any agent: `LevelSolvability::isPathReachable` certifies that walking and jumping can connect the start column to the goal, so an unplayable

candidate is rejected without spending an episode on it.

That oracle is also the one piece of this side project that has already paid rent on the main line, and it is the model for how the rest should behave if it ever does. The original branch built a full oracle — per-frame ballistic simulation plus a bottleneck-cost search that grades difficulty. What came back to `dev` is a deliberately simplified, dependency-free reimplementaion: a column-reachability search bounded by the game's own walk, run and jump constants, keeping the question the project actually needs answered ("is a required move impossible at all?") and dropping the one it does not ("how hard is the hardest required move?"). The header says so in as many words. Cherry-picking one honest reimplementaion is the whole of what these branches owe the submission; the rest stays where it is.

14.1.3 One engine horizon: two players over a network

Not delivered — **a direction, with no branch.**

Two-player is local today: both players share one process, one keyboard and one camera. Putting them on two machines is a genuine horizon rather than a task, and the reason is a decision the project already made for other purposes. `ReplayRecorder` records a stream of `GameSnapshot state`, not a stream of inputs, and the header explains why: the simulation is not deterministic, so re-running the inputs would not reproduce the run. That single fact rules out the cheap netcode — lockstep input exchange, which is what a deterministic simulation buys you — and points at authoritative state replication instead, which is the shape the existing snapshot stream already has. What is missing is a transport, a decision about who simulates authoritatively, delta encoding (a snapshot is a whole entity list, sent whole), and client-side reconciliation for the latency that remains. Each of those is a project, and naming them is more useful than promising the feature.

Which is the point of listing only three horizons. Everything here is a direction with its first real obstacle named, on the reasoning that a future-work section whose items have no known obstacles has not been thought about yet.

15 References

- Repository: github.com/ndmhuy/SuperMarioGame

- SPEC.md — frozen behavioural specification (constants, schemas, entity rules)
- AGENTS.md — engineering rules, and the incident history behind each
- docs/learning/mid-frame-entity-spawn-crash.html — full analysis of §10.1
- SFML 3.0.2 — sfml-dev.org; Dear ImGui + ImGui-SFML; nlohmann/json
- Gamma, Helm, Johnson, Vlissides, *Design Patterns* (1994) — Factory, Singleton, Observer, State, Strategy, Command, Decorator, Memento, Template Method
- Nystrom, *Game Programming Patterns* — Object Pool, Game Loop, Component
- Fiedler, "Fix Your Timestep!" — the fixed-step accumulator in §8.1
- Assets are from *Super Mario Bros.* (Nintendo, 1985), used for non-commercial educational purposes only. All rights remain with their owners.

16 Appendix

16.1 Build and run

```
cd SuperMarioGame
mkdir -p build && cd build
cmake ..                # fetches SFML, ImGui-SFML and nlohmann/json
cmake --build . -j8
ctest --output-on-failure # 32 verification targets registered as CTest cases
./SuperMarioGame
```

16.2 Controls

Action	Player 1	Player 2
Move	A / D	← / →
Jump	W or Space	↑ or Right Shift
Crouch / ground pound	S	↓
Run (hold)	Left Shift	N
Fireball	F	M

Action	Player 1	Player 2
Rewind time (hold)	R	—
Level editor	F1	—
Dev panel / debug console	F12 / ~	—
Pause	Escape	Escape

16.3 Figures at a glance

Measure	Value
C++ source files / headers	134 / 145
Lines of C++ (excluding third-party)	36,766
Concrete entity classes	5 players, 13 enemies, 13 items, 10 blocks
Design patterns	see §7 for the full list, each with the problem it solves
Verification harnesses	40 source files, 38 built, 32 registered as CTest cases
Commits on the integration branch	513
Sound effects / music tracks	29 / 12
Achievements	12

16.4 UML diagrams, collected

Every class diagram in this report, in one place, at full size — not a redrawn summary, the same figures §6 discusses in context, collected here for a reader who wants the whole structure in one pass rather than hunting back through the prose. Each is regenerated from the current headers at build time by the same `gen_class_diagram.py` pass that draws the inline copies (§6's opening note), so this appendix cannot go stale relative to them the way a hand-drawn "master diagram" done once and forgotten would.

Seven of the nine show **every attribute and method, with its real visibility** — a classic three-compartment UML box, not just a name in a box — specifically so encapsulation is something a reader can verify by eye (how much of each box is `-/#` versus `+`) rather than take §6's prose about it

on faith. Static members are underlined and pure-virtual operations italicised, both standard UML notation; the same dashed border and `«stereotype»` tag from §6 still mark an abstract class or interface. Two roots (`Entity`, `Character`) are kept to the compact, name-only form instead: both contain `Player` (109 members) or `PlayingState` (132), and a full listing for either one alone makes the whole diagram many times taller than every other class on it combined — measured at roughly seven times the width in height before this exception was made, against one-to-two times for every other group here. Their full member lists are not missing, only shown a different way: `Entity`’s own branches (`Item`, `Block`, `Enemy`) and `Character`’s own children (`Enemy`, and `Player`’s siblings inside it) are the very next diagrams, each in full detail.

Figure	Root class	Discussed in	Detail	What it shows
6	<code>Entity</code>	§6.1	compact	The four branches under the abstract root: <code>Character</code> , <code>Item</code> , <code>Block</code> , <code>Projectile</code> .
6a	<code>Item</code>	§6.2	full	Thirteen concrete power-ups and pickups behind one <code>activate()</code> call.
6b	<code>Block</code>	§6.2	full	Ten concrete blocks; <code>FallingPlatform</code> drives itself through a phase enum (see §7.4 — not the State pattern). <code>Thwomp</code> is an <code>Enemy</code> , not a <code>Block</code> .
7	<code>Character</code>	§6.3	compact	<code>Player</code> and <code>Enemy</code> split, including the Shadow-Mario replay rival.
8	<code>Enemy</code>	§6.3	full	The enemy branch, with the Boss sub-hierarchy sealed by Template Method.
9	<code>IPlayerState</code>	§6.4	full	Five player forms plus the two Decorators that wrap them.
10	<code>IGameState</code>	§6.5	full	9 screens behind one State interface.
11	<code>ICommand</code>	§6.5	full	Input as objects — the Command pattern’s own hierarchy.
12	<code>IMovementStrategy</code>	§6.5	full	Eight interchangeable enemy movements — the Strategy pattern’s own hierarchy.

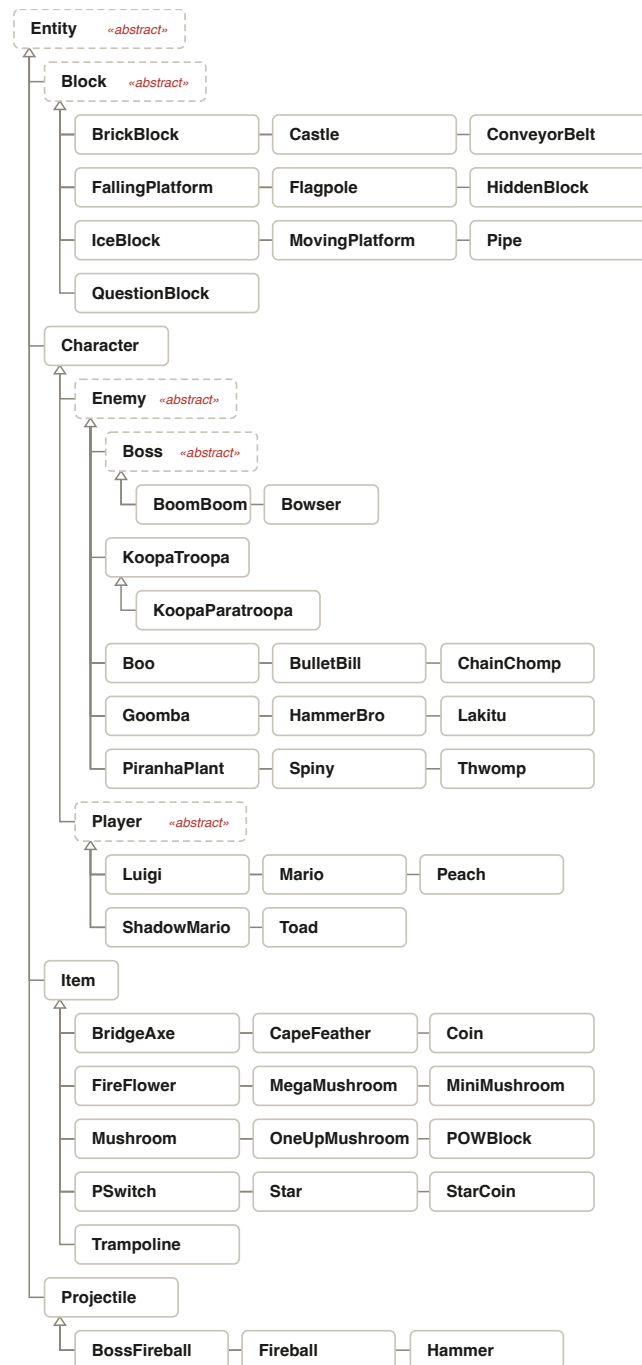


Figure 6 (full tree, compact boxes) — The complete Entity hierarchy.

Every concrete Entity subclass the game has, four branches deep from one abstract root. Compact rather than detailed — see the note above the index table.

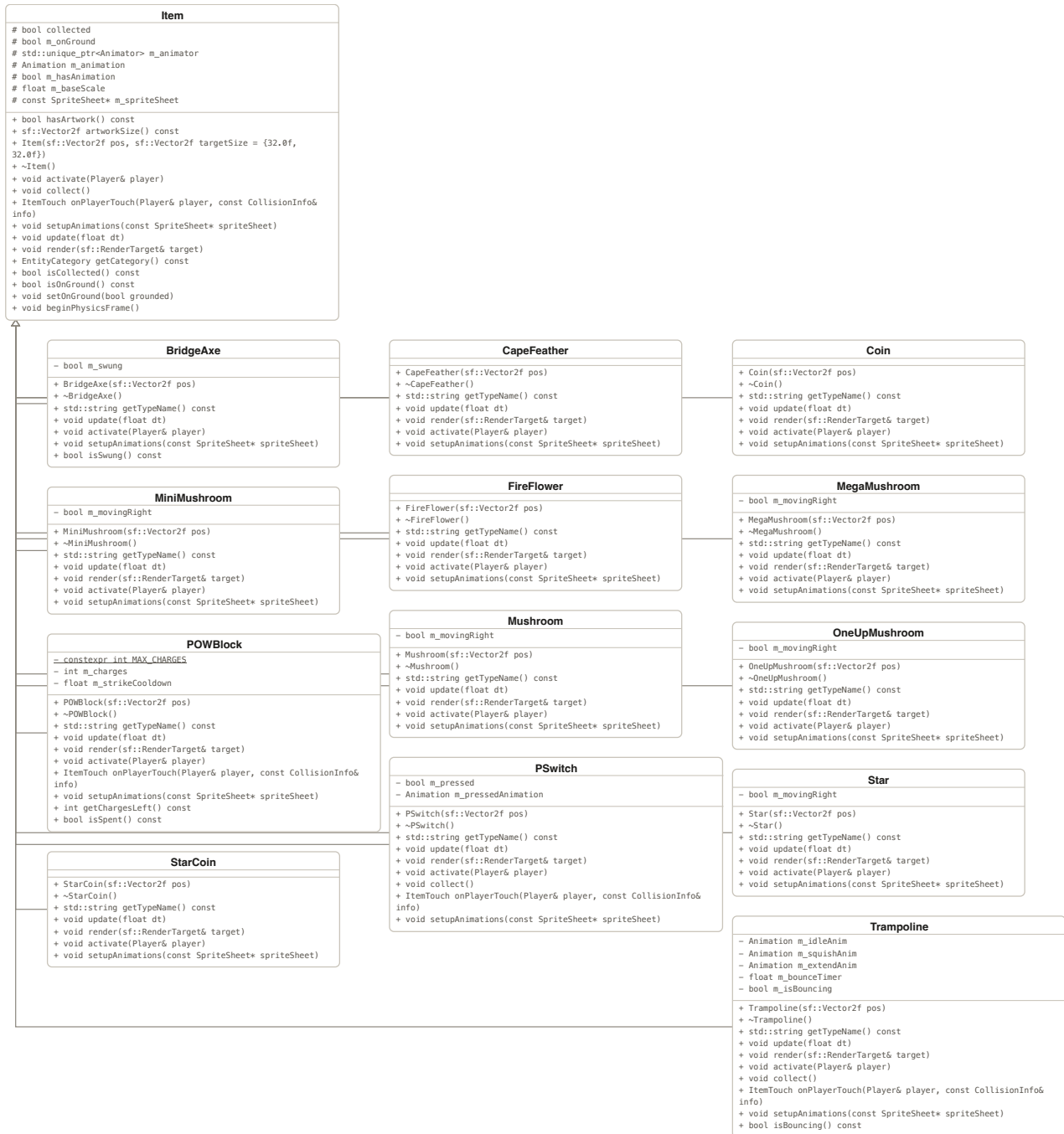


Figure 6a (full detail) — The complete Item hierarchy, every attribute and method.

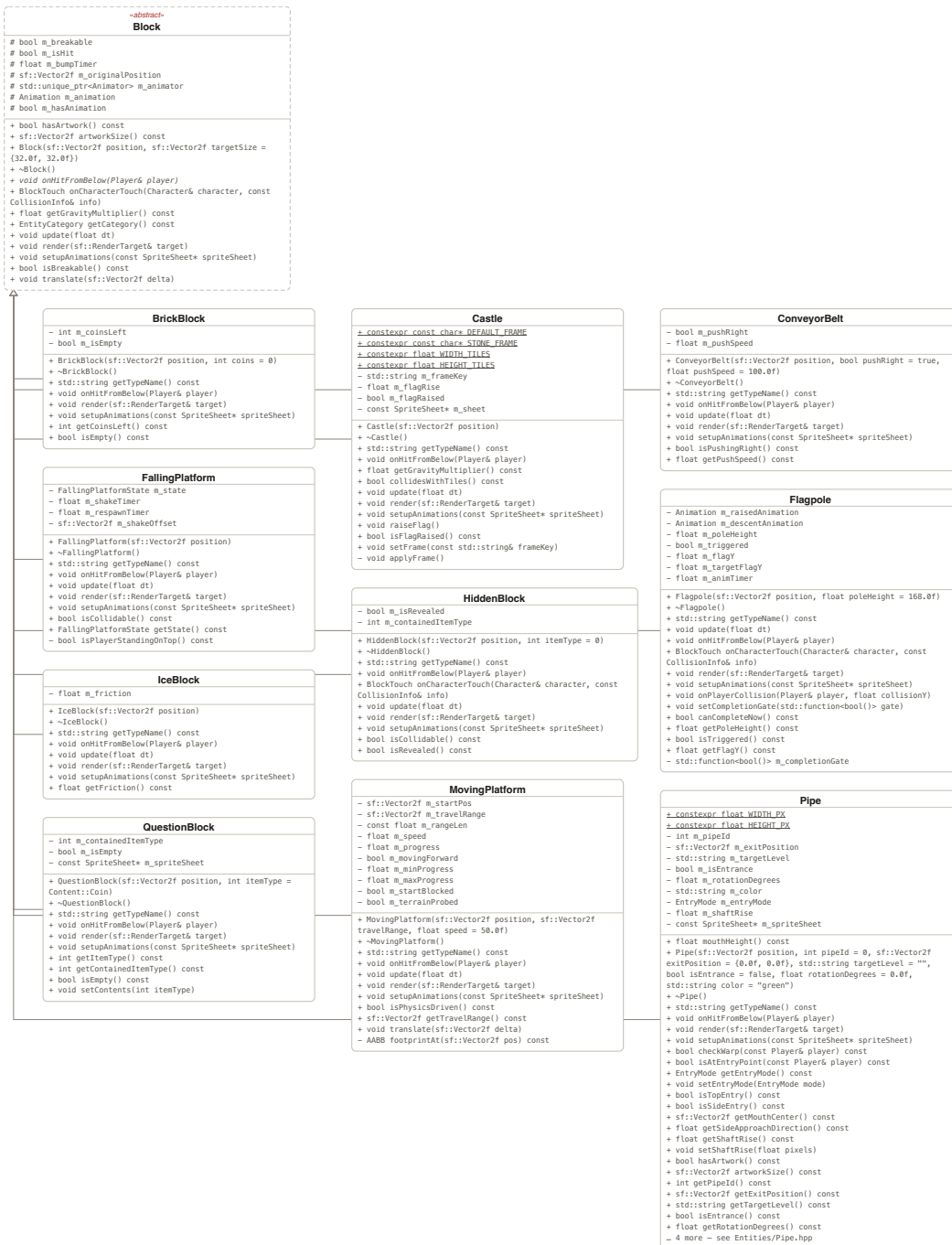


Figure 6b (full detail) — The complete Block hierarchy, every attribute and method.

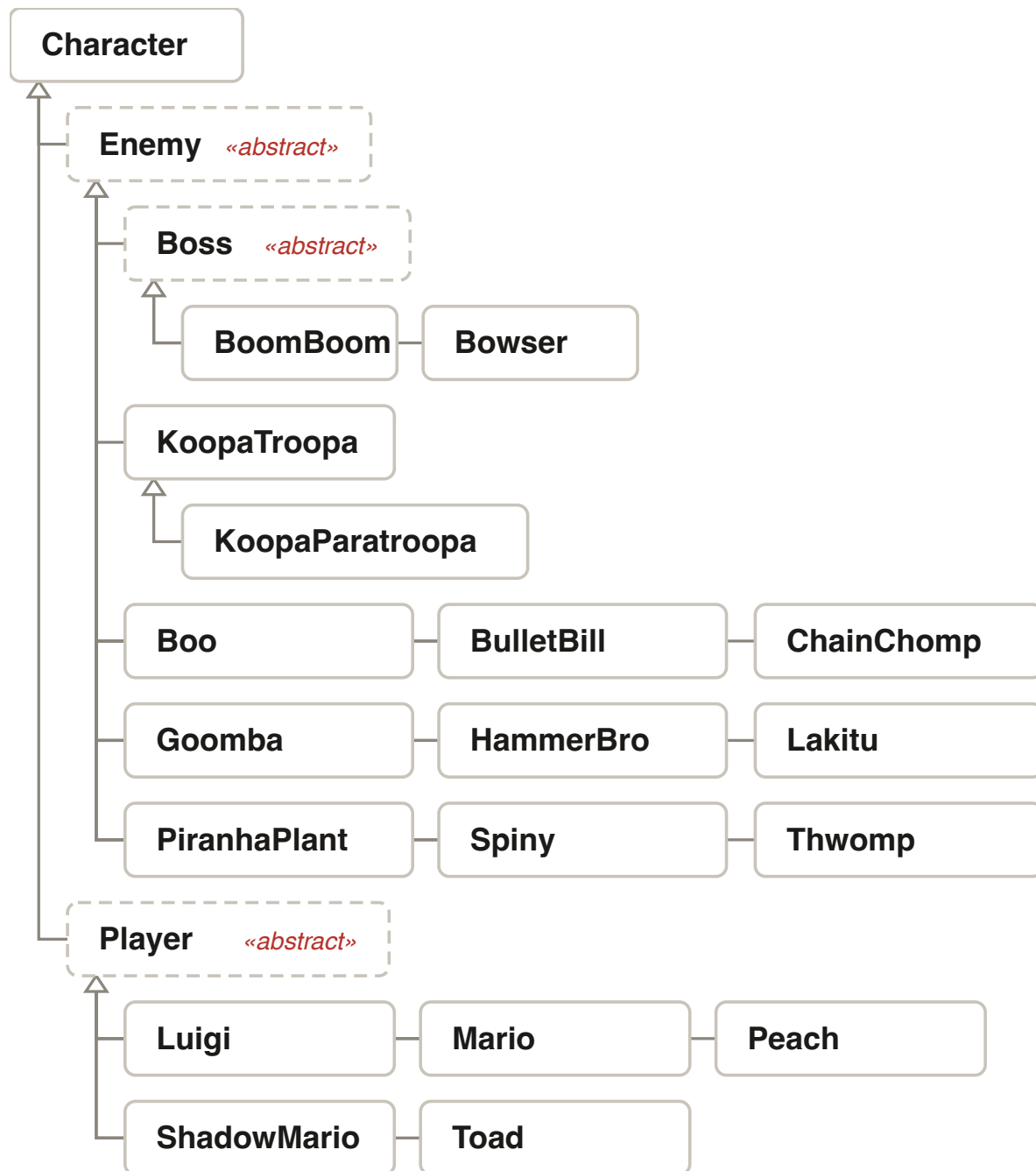


Figure 7 (full tree, compact boxes) — The complete Character hierarchy.

Every playable character and every enemy, including the Boss sub-branch, in one tree. Compact rather than detailed for the same reason as Figure 6.

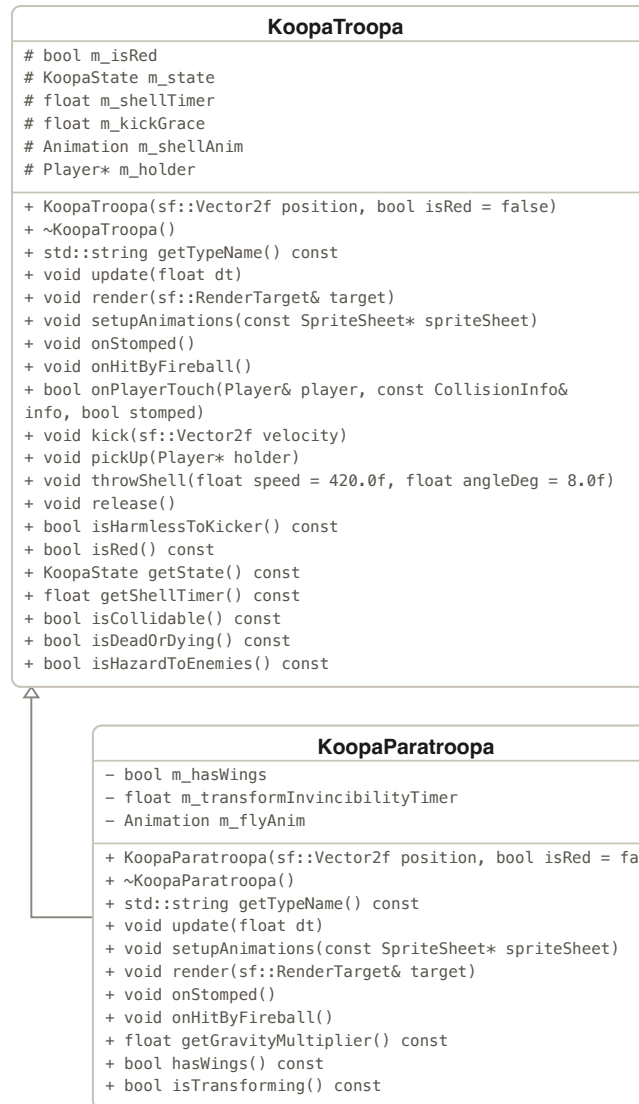
«abstract»

Enemy

```
# std::unique_ptr<IMovementStrategy> m_aiStrategy
# int m_scoreValue
# bool m_isFlipped
# bool m_isDyingDownward
# std::unique_ptr<Animator> m_animator
# Animation m_animation
# bool m_hasAnimation

+ bool hasArtwork() const
+ sf::Vector2f artworkSize() const
+ Enemy(sf::Vector2f position, int scoreValue = 100,
sf::Vector2f targetSize = {32.0f, 32.0f})
+ ~Enemy()
+ void update(float dt)
+ void render(sf::RenderTarget& target)
+ void setupAnimations(const SpriteSheet* spriteSheet)
+ void setStrategy(std::unique_ptr<IMovementStrategy>
strategy)
+ IMovementStrategy* getStrategy() const
+ void translate(sf::Vector2f delta)
+ void onStomped()
+ void onHitByFireball()
+ bool onPlayerTouch(Player& player, const CollisionInfo&
info, bool stomped)
+ bool isHazardToEnemies() const
+ EntityCategory getCategory() const
+ int getScoreValue() const
+ void setScoreValue(int value)
+ void applySpeedScale(float scale)
+ void setSpeed(float newSpeed)
+ bool hasFloorAhead(float dx) const
+ bool isFlipped() const
+ bool isDyingDownward() const
+ bool isDeadOrDying() const
+ void triggerFlipDeath(sf::Vector2f launchVel = {80.0f,
-250.0f})
... 6 more – see Entities/Enemy.hpp
```





Boo
- Animation m_seenAnim - Animation m_moveAnim
+ Boo(sf::Vector2f position) + ~Boo() + std::string getTypeName() const + void update(float dt) + void render(sf::RenderTarget& target) + void setupAnimations(const SpriteSheet* spriteSheet) + void onStomped() + void onHitByFireball() + float getGravityMultiplier() const + bool collidesWithTiles() const

HammerBro
+ HammerBro(sf::Vector2f position) + ~HammerBro() + std::string getTypeName() const + void setupAnimations(const SpriteSheet* spriteSheet) + void onStomped() + void onHitByFireball()

Spiny
- bool m_isEgg - Animation m_eggAnim - <u>std::vector<Spiny*> s_live</u>
+ Spiny(sf::Vector2f position, bool isEgg = true) + ~Spiny() + <u>int liveCount()</u> + std::string getTypeName() const + void onStomped() + void onHitByFireball() + void update(float dt) + void setupAnimations(const SpriteSheet* spriteSheet) + bool isCollidable() const + bool isEgg() const + void setEgg(bool isEgg)

Goomba
- bool m_isRed - bool m_isSquished - float m_squishTimer - Animation m_squishAnim
+ Goomba(sf::Vector2f position, bool isRed = false) + ~Goomba() + std::string getTypeName() const + void update(float dt) + void render(sf::RenderTarget& target) + void setupAnimations(const SpriteSheet* spriteSheet) + void onStomped() + void onHitByFireball() + bool isRed() const + bool isSquished() const + float getSquishTimer() const + const AABB& getBoundingBox() const + bool isDeadOrDying() const + bool collidesWithTiles() const

Lakitu
+ <u>constexpr int MAX_SPINIES</u> + <u>constexpr float FLOWER_DROP_INTERVAL</u> - float m_eggTimer - int m_spawnCount - float m_flowerTimer - int m_flowerCount
+ Lakitu(sf::Vector2f position) + ~Lakitu() + std::string getTypeName() const + void onStomped() + void onHitByFireball() + void update(float dt) + void setupAnimations(const SpriteSheet* spriteSheet) + float getGravityMultiplier() const + bool collidesWithTiles() const + float getEggTimer() const + int getSpawnCount() const + float getFlowerTimer() const + int getFlowerCount() const + bool isEngaged() const + <u>constexpr float ENGAGE_RANGE_X =</u> static_cast<float>(Constants::WINDOW_WIDTH) - bool dropFireFlower()

BulletBill

```
+ BulletBill(sf::Vector2f position, float dirX = -1.0f)
+ ~BulletBill()
+ std::string getTypeName() const
+ void setupAnimations(const SpriteSheet* spriteSheet)
+ void onStomped()
+ void onHitByFireball()
+ float getGravityMultiplier() const
+ bool collidesWithTiles() const
```

ChainChomp

```
+ ChainChomp(sf::Vector2f position)
+ ~ChainChomp()
+ std::string getTypeName() const
+ void setupAnimations(const SpriteSheet* spriteSheet)
+ void onStomped()
+ void onHitByFireball()
+ bool onPlayerTouch(Player& player, const CollisionInfo&
info, bool stomped)
```

PiranhaPlant

```
+ PiranhaPlant(sf::Vector2f position)
+ ~PiranhaPlant()
+ std::string getTypeName() const
+ void setupAnimations(const SpriteSheet* spriteSheet)
+ void onStomped()
+ void onHitByFireball()
+ void render(sf::RenderTarget& target)
+ bool isCollidable() const
+ float artworkVisibleHeight() const
+ float getGravityMultiplier() const
+ bool collidesWithTiles() const
- float emergedHeight() const
```

Thwomp

```
- Animation m_dormantAnim
- Animation m_activeAnim

+ Thwomp(sf::Vector2f position)
+ ~Thwomp()
+ std::string getTypeName() const
+ void update(float dt)
+ void setupAnimations(const SpriteSheet* spriteSheet)
+ void onStomped()
+ void onHitByFireball()
+ float getGravityMultiplier() const
```

Figure 8 (full detail) — The complete Enemy/Boss hierarchy, every attribute and method.

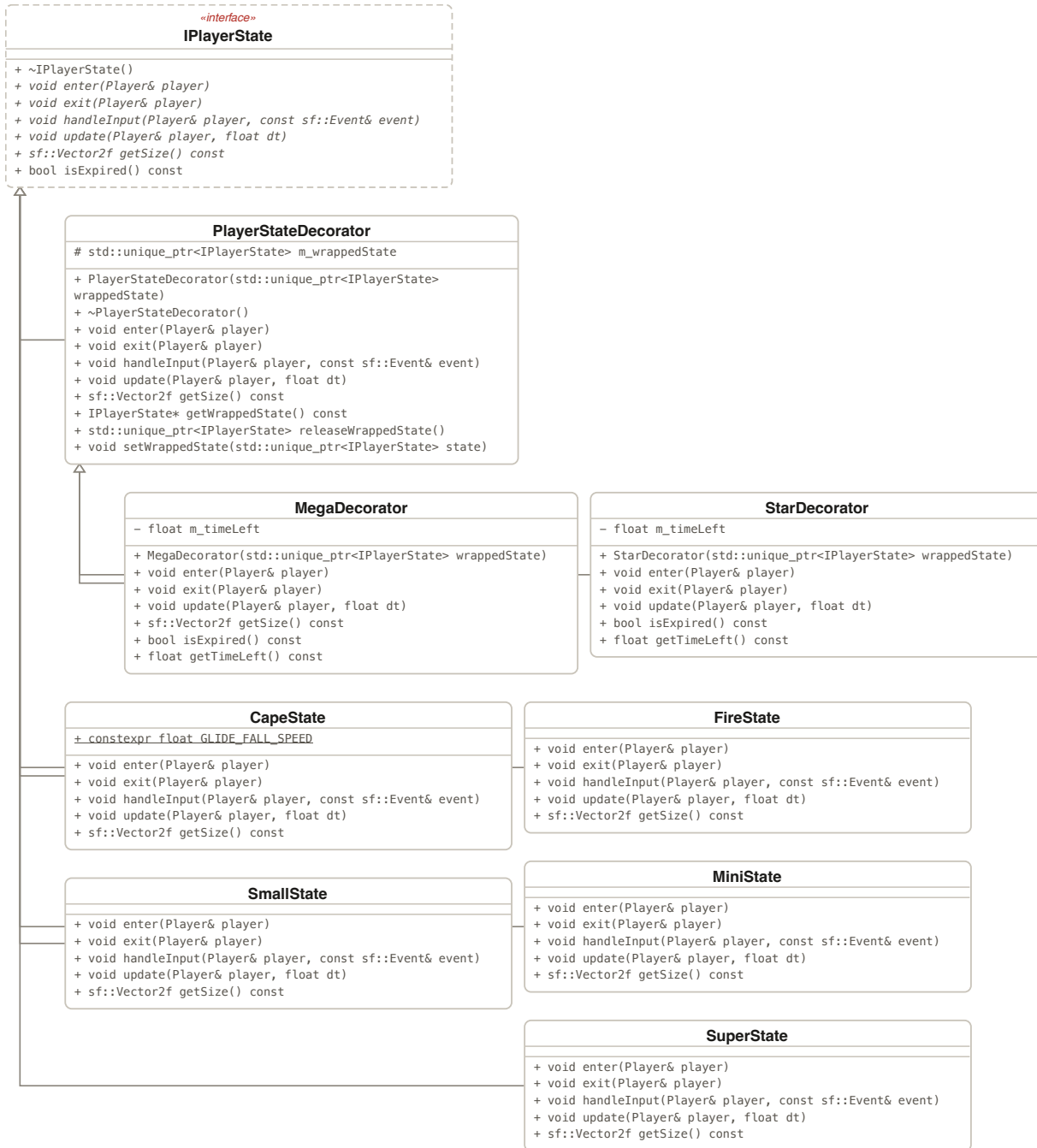
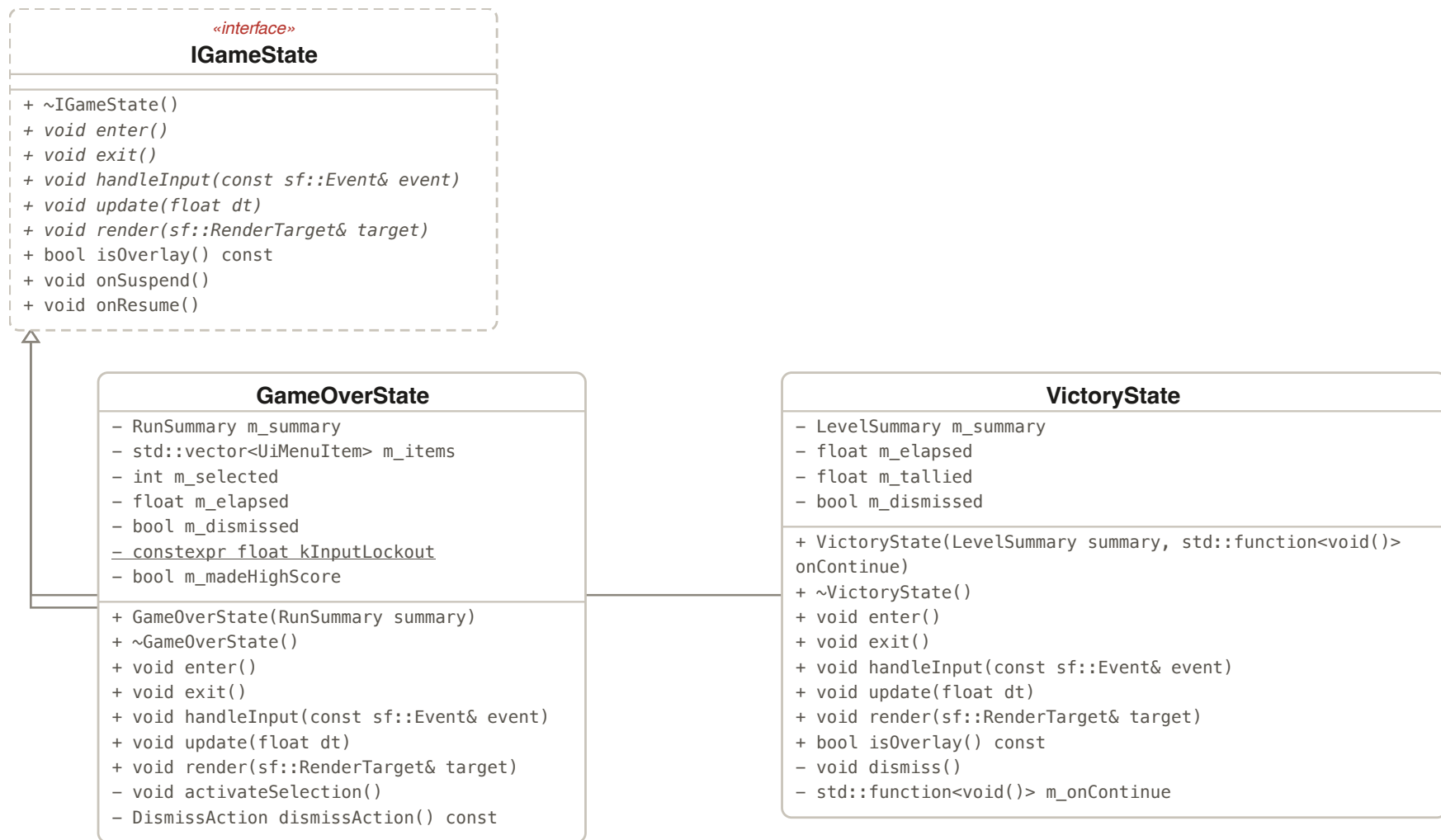


Figure 9 (full detail) — The complete player-form hierarchy, every attribute and method.



PlayingState
<pre> - PhysicsEngine m_physicsEngine - TileMap m_tileMap - std::vector<std::unique_ptr<Entity>> m_entities - std::vector<std::unique_ptr<Entity>> m_pendingSpawns - MapEditor m_mapEditor - EventBus::ScopedSubscription m_checkpointSub - EventBus::ScopedSubscription m_playerDiedSub - EventBus::ScopedSubscription m_powerUpSub - EventBus::ScopedSubscription m_levelCompleteSub - EventBus::ScopedSubscription m_entitySpawnSub - EventBus::ScopedSubscription m_fireballSub - float m_warpCooldown - PipeEntry m_pipeEntry = constexpr float DEATH_FALL_SECONDS = constexpr float DEATH_FALL_MINIMUM - DeathState m_death - DeathState m_death2 - Player* m_player - Player* m_player2 - std::unique_ptr<AIController> m_aiController - ShadowMario* m_shadow - MatchConfig m_match - int m_selectedCharIndex - int m_selectedLevelIndex _ 78 more - see Core/PlayingState.hpp </pre>
<pre> + PlayingState(bool startInEditor = false, bool isProcedural = false, const MapGeneratorConfig& genConfig = MapGeneratorConfig(), int characterIndex = 0, int levelIndex = 0, MatchConfig match = MatchConfig{}, bool isEndless = false, int pendingLoadSlot = 0, bool isAttractDemo = false, std::string customLevelPath = std::string(), bool isPlaytest = false) + ~PlayingState() + void enter() + void exit() + void handleInput(const sf::Event& event) + void update(float dt) + void render(sf::RenderTarget& target) + void onSuspend() + void onResume() - void regenerateProceduralLevel() - void saveToSlot(int slot) - void loadFromSlot(int slot) - void beginPipeEntry(sf::Vector2f mouthCenter, float approachX, const std::string& targetLevel, sf::Vector2f exit) - void updatePipeEntry(float dt) - void rememberEliminatedIdentity(const Player* who, DeathState& death) const - void applyEliminatedBadges(HudData& hudData) const - DeathState* deathStateFor(const Player* who) - bool anyDeathInProgress() const - void updateDeathSequence(float dt) - void respawnPlayer(Player* who) - void spawnMatchParticipants() - void updateShadow(float dt) - float shadowProximitySeconds() const - void updateVersusCamera(float dt) _ 56 more - see Core/PlayingState.hpp </pre>

EditorState
<pre> - constexpr float MENU_BAR_H - constexpr float LEFT_W - constexpr float RIGHT_W - constexpr float BOTTOM_H - EditorState& m_owner - TileMap m_tileMap - std::vector<std::unique_ptr<Entity>> m_entities - Camera m_camera - MapEditor m_mapEditor - EditorBridge m_bridge - std::unique_ptr<SpriteSheet> m_playerSheet - std::unique_ptr<SpriteSheet> m_enemySheet - std::unique_ptr<SpriteSheet> m_itemSheet - std::unique_ptr<SpriteSheet> m_scenerySheet - BackgroundRenderer m_background - TileMapRenderer m_tileMapRenderer - EntityArtBinder m_artBinder - Document m_document - std::string m_pendingOpenPath - float m_animTimer - bool m_suspended - bool m_showOpen - bool m_showNew - bool m_showSaveAs ... 14 more — see Core/EditorState.hpp </pre>
<pre> + EditorState(std::string openPath = std::string()) + ~EditorState() + void enter() + void exit() + void handleInput(const sf::Event& event) + void update(float dt) + void render(sf::RenderTarget& target) + void onSuspend() + void onResume() + EditorBridge(EditorState& owner) : m_owner(owner) + void admit(Entity* entity) + void release(Entity* entity) - void admitEntity(Entity* entity) - void forgetEntity(Entity* entity) - void newLevel(int width, int height, const std::string& theme, const std::string& name) - bool openLevel(const std::string& path) - bool saveTo(const std::string& path) - void save() - void saveAsCurrentName() - void playtest() - void leaveToMenu() - void syncCameraBounds() - void showLevelOrigin() - void setStatus(std::string message) ... 8 more — see Core/EditorState.hpp </pre>

MenuState
- Page m_page - int m_mainSelected - int m_genSelected - int m_mpSelected - int m_loadSelected - int m_customSelected - float m_elapsed - float m_idleTime - std::array<SaveSlotPreview, 3> m_slotPreviews - int m_pendingDeleteSlot - int m_deleteConfirmSelected - MatchConfig m_match - bool m_dismissed - MapGeneratorConfig m_generatorConfig - int m_selectedThemeIdx - int m_selectedDifficultyIdx - std::vector<UiMenuItem> m_mainItems - std::unique_ptr<SpriteSheet> m_playerSheet - std::unique_ptr<SpriteSheet> m_scenerySheet - BackgroundRenderer m_background - float m_cloudScroll - float m_walkerX
+ MenuState() + ~MenuState() + void enter() + void exit() + void handleInput(const sf::Event& event) + void update(float dt) + void render(sf::RenderTarget& target) - void moveSelection(int delta) - void adjustSelection(int direction) - void activateSelection() - void applyDifficultyPreset(int index) - void drawBackground(sf::RenderTarget& target) const - bool isMultiplayerRowEnabled(MpRow row) const - std::vector<UiMenuItem> buildMultiplayerItems() const - void refreshSlotPreviews() - std::vector<UiMenuItem> buildLoadItems() const - void requestSlotDelete() - void performSlotDelete() - std::vector<UiMenuItem> buildCustomLevelItems() const

PauseState
- std::string m_notice - float m_noticeTimer - std::vector<UiMenuItem> m_items - int m_selected - float m_elapsed - Page m_page - std::array<SaveSlotPreview, 3> m_slotPreviews - int m_slotSelected - int m_pendingSlot - int m_confirmSelected - bool m_dismissed
+ PauseState(std::function<void()> onRestartLevel, std::function<void()> onSaveGame, std::function<void()> onQuitToMenu) + ~PauseState() + void enter() + void exit() + void handleInput(const sf::Event& event) + void update(float dt) + void render(sf::RenderTarget& target) + bool isOverlay() const - void activateSelection() - void moveSelection(int delta) - void activateMenuSelection() - void openSlotPicker() - void refreshSlotPreviews() - std::vector<UiMenuItem> buildSlotItems() const - void activateSlotSelection() - void activateConfirmSelection() - void commitSaveTo(int slot) - void renderMenuPage(sf::RenderTarget& target) const - void renderSlotPickerPage(sf::RenderTarget& target) const - void renderConfirmPage(sf::RenderTarget& target) const - std::function<void()> m_onRestartLevel - std::function<void()> m_onSaveGame - std::function<void()> m_onQuitToMenu

CharacterSelectState

```
- std::vector<CharacterSlot> m_slots
- int m_selected
- float m_elapsed
- bool m_dismissed
- bool m_startInEditor
- bool m_isProcedural
- MapGeneratorConfig m_genConfig
- std::unique_ptr<SpriteSheet> m_playerSheet

+ CharacterSelectState(bool startInEditor = false, bool
isProcedural = false, const MapGeneratorConfig& genConfig =
MapGeneratorConfig())
+ ~CharacterSelectState()
+ void enter()
+ void exit()
+ void handleInput(const sf::Event& event)
+ void update(float dt)
+ void render(sf::RenderTarget& target)
- void moveSelection(int delta)
- void confirmSelection()
- bool isUnlocked(const CharacterSlot& slot) const
```

WorldMapState

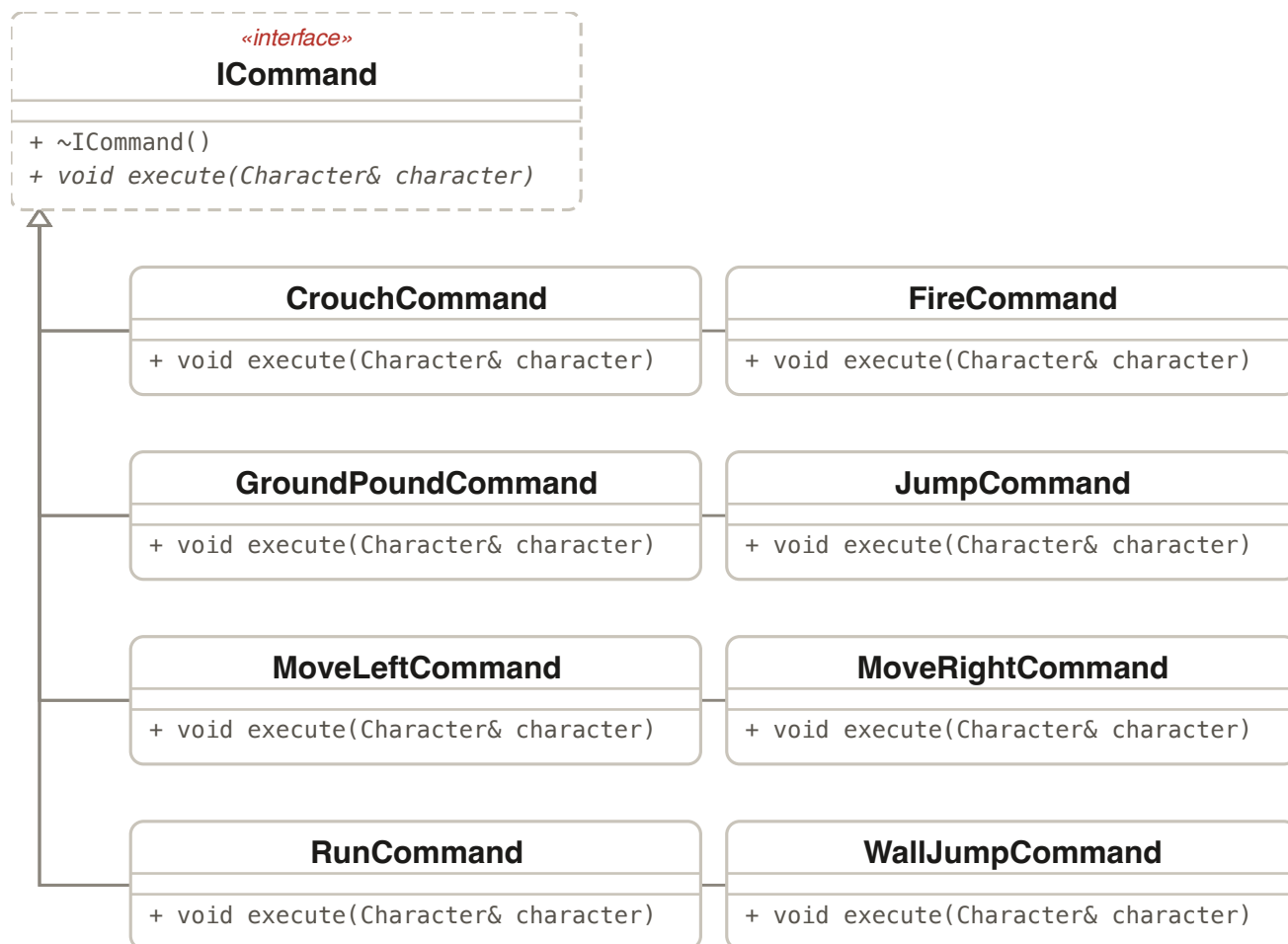
```
- int m_characterIndex
- int m_selected
- float m_elapsed
- bool m_dismissed
- std::vector<Node> m_nodes
- std::vector<LevelProgress> m_progress
- std::unique_ptr<SpriteSheet> m_scenerySheet
- std::unique_ptr<SpriteSheet> m_playerSheet

+ WorldMapState(int characterIndex = 0)
+ ~WorldMapState()
+ void enter()
+ void exit()
+ void handleInput(const sf::Event& event)
+ void update(float dt)
+ void render(sf::RenderTarget& target)
- void moveSelection(int delta)
- void confirmSelection()
- void buildNodes()
- void drawPath(sf::RenderTarget& target) const
- void drawNode(sf::RenderTarget& target, const Node& node)
const
```

OptionsState

```
- Page m_page
- std::vector<Row> m_rows
- int m_selected
- float m_elapsed
- int m_awaitingBindingRow
- std::string m_notice
- float m_noticeTimer
- std::vector<HighScoreEntry> m_highScores
- int m_achievementScroll
- bool m_closing

+ OptionsState()
+ OptionsState(Page startPage) : m_page(startPage)
+ ~OptionsState()
+ void enter()
+ void exit()
+ void handleInput(const sf::Event& event)
+ void update(float dt)
+ void render(sf::RenderTarget& target)
+ bool isOverlay() const
- void buildRows()
- bool isRowPage() const
- void moveRow(int delta)
- void adjustSelected(int direction)
- void activateSelected()
- std::string valueTextFor(const Row& row) const
- void close()
- void renderStatisticsPage(sf::RenderTarget& target) const
- void renderAchievementsPage(sf::RenderTarget& target) const
```


Figure 10 (full detail) — The complete screen/State hierarchy, every attribute and method.*Figure 11 (full detail) — The complete Command hierarchy, every attribute and method.*

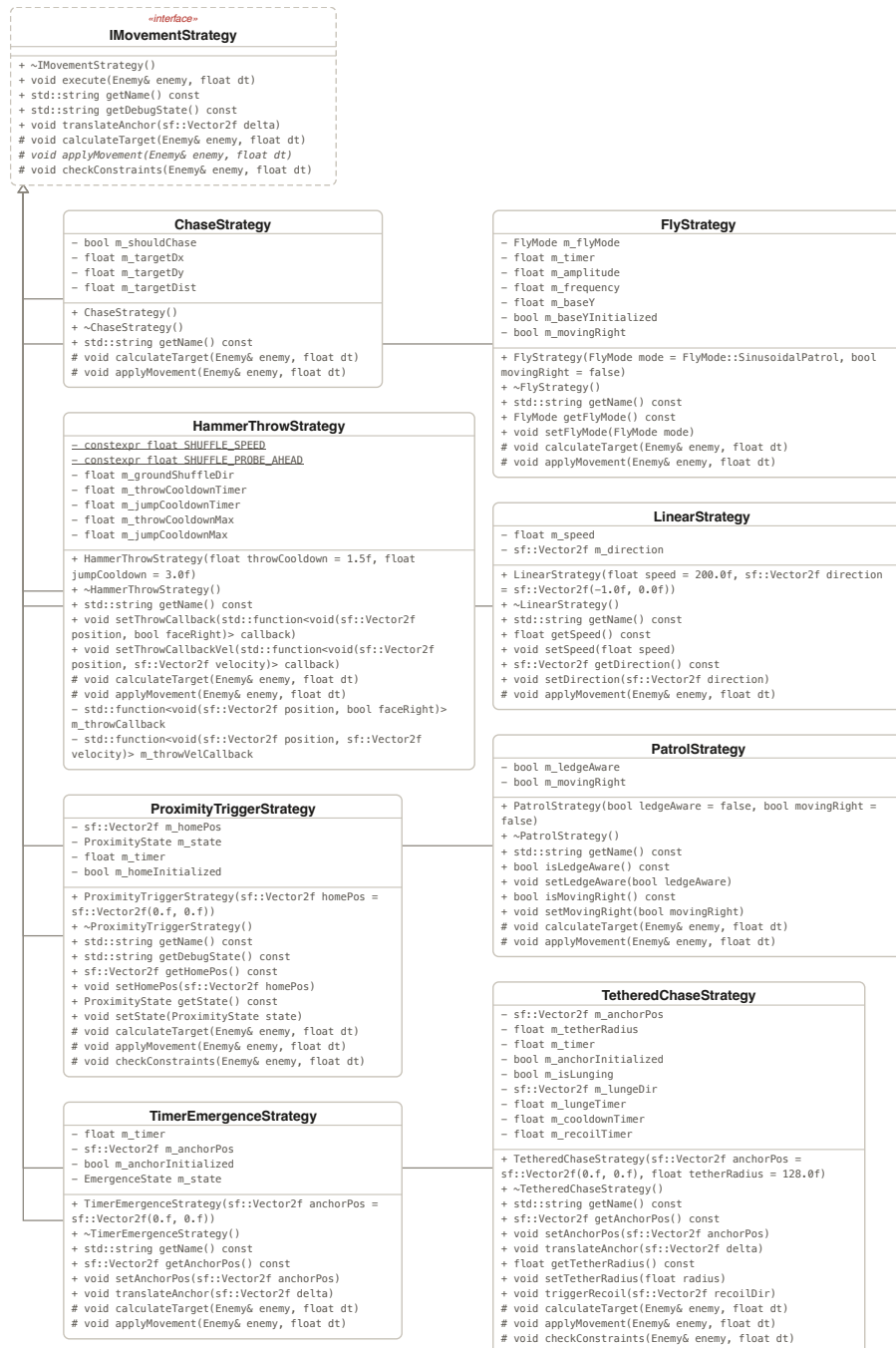


Figure 12 (full detail) — The complete Strategy hierarchy, every attribute and method.

Group 52 · CS202 Object-Oriented Programming · University of Science, VNU-HCM

Generated by `reports/build_report.py` from the repository at commit 3ec20ac. Self-contained:
opens from disk, no network, prints to PDF.